University of Northampton

الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري
Arab Academy for Science, Technology & Maritime Transport

TAKHLEES

By

221005109 Ahmed Mohamed Abdelfadil

221005621 Seif Amr Attia

221006191 Marwan Mohamed Salah

221006916 Seif Eldin Hassan Mohamed

Submitted in partial fulfilment of the requirements for accomplishing

[ELC479E] Computer Applications in Business

Business Information Systems Department

College of Management & Technology

Arab Academy for Science, Technology & Maritime Transport

[2025 – 2026]

This page intentionally left blank . . .

Table of Contents

This page intentionally left blank . . .

List of Figures

This page intentionally left blank . . .

List of Tables

This page intentionally left blank . . .

Acknowledgements

We would like to express our deepest gratitude to those who contributed to the success of this graduation project, Takhlees.

Firstly, we extend our heartfelt thanks to Professor Dr. Amr A.Hamid for his invaluable guidance, profound insights, and unwavering support throughout the journey. His expertise and dedication have been a source of inspiration, helping us navigate complex challenges and achieve our goals. We are also deeply thankful to TA Dr. Malak for her helpful feedback and constant support, which played a key role in shaping our work. Their mentorship has been instrumental in shaping this project into its final form. We are truly fortunate to have had the privilege of learning from such esteemed professionals.

In addition, we are profoundly grateful to all the Professors and TAs of the BIS department who shared their knowledge, offered guidance, and supported us throughout our academic journey. Their contributions have been essential to the successful completion of this project.

This page intentionally left blank . . .

Chapter 1: Introduction

In today's global trade environment, customs clearance plays a crucial role in the import and export process.

Many individuals and businesses in Egypt face difficulties finding reliable customs clearance companies that suit their specific needs whether they are importing personal items, vehicles, or commercial goods.

This project aims to develop a web platform that connects customers with certified customs clearance companies efficiently. The system allows users to browse, compare, and communicate with clearance agents, based on category, service type, ratings, and price.".

1.1 BACKGROUND

The customs clearance process in Egypt can be complicated and time-consuming for both individuals and businesses, most importers struggle to find trusted customs clearance companies, understand the required documents, or estimate the expected cost and timeframes.

1.2 PURPOSE

In this project, we will develop "Takhlees", a web-based intermediary platform designed to streamline and digitize the bureaucratic process of finishing papers related to ports and government entities in Egypt. The system will bridge the gap between individual importers and professional customs clearance agencies, initially covering the Automotive and Clothing sectors, with a scalable architecture that allows for the addition of new service categories at any time.

1.3 REPORT OUTLINE

In Chapter 2 of this report, we will focus on Database table entities and Entity Relationship Diagram (ERD) and illustrate the backend design in Node.js and how we implement REST APIs.

This page intentionally left blank . . .

Chapter 2: Takhlees Backend Design

2.1 DATABASE DESIGN

Figure 1: ERD

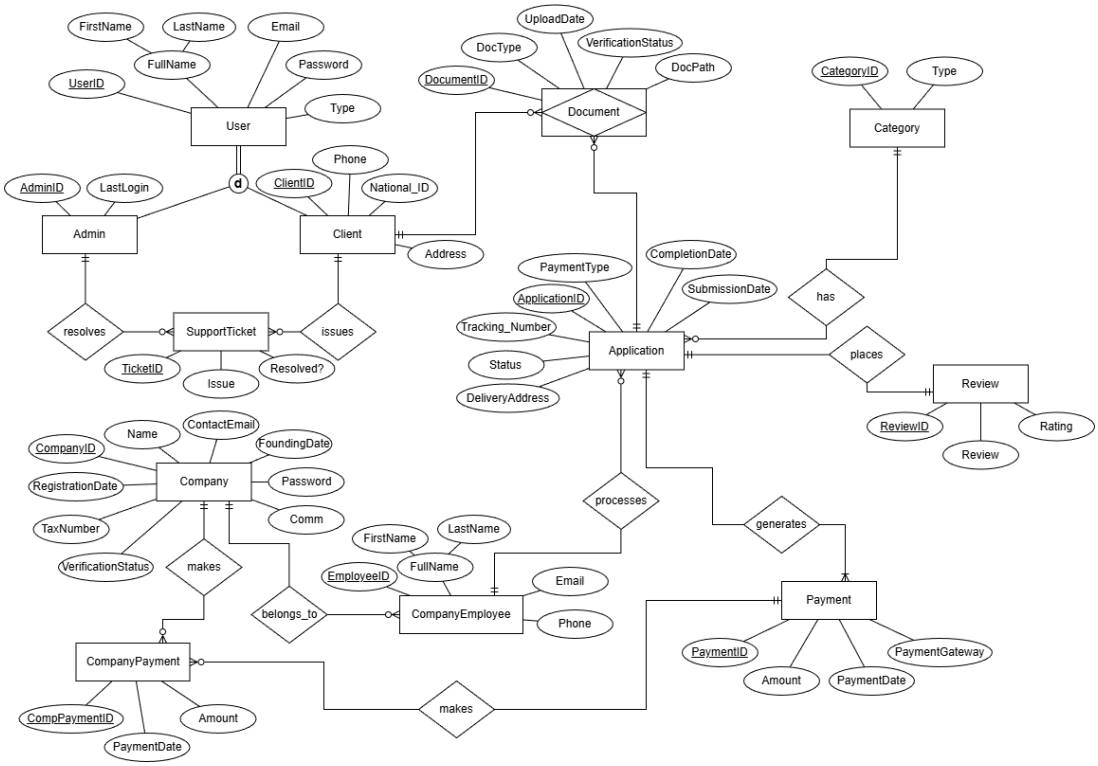


Table Entities & Attributes:

No.	Entity Name	Descibtion
1	User	Stores information about the app users, including login credentials and profile details
2	Admin	Stores administrator accounts and their last login time
3	Client	Stores client profiles, and related data
4	Document	Contains files or documents uploaded by the user related to application
5	Application	Records information about applications submitted by clients
6	Support ticket	Tracks client support requests and their statuses
7	Company payment	Stores records of payments made by companies for system commission and services
8	Payment	Tracks individual payment transactions, including amount, date, and method
9	Company Employee	Stores employee details linked to a company account

10	Category	Contains company profiles and related business information
11	Review	Stores client reviews and ratings for services
12	Company	Contains company profiles and related business information

Table 1: Database Entities

Figure 2: User Table

UserID	FirstName	LastName	Email	Password	Type
1	Adham	Foda	Adhamfoda@example.com	adhamfoda123@	A
3	khaled ahmed mohamed	ahmed mohamed assss	ahmedkhaledmoamed@gmail.com	ahmed123@	C
11	Ahmed	Mohamed	ahmed.mohamed@gmail.com	ahmed1234	C
12	omar	khaled	omarkhaled@gmail.com	omar123@	C
13	Ahmed	Hassa	Ahmedhassan@gmail.com	ahmed123@	C
14	Ahmed	Hassa	Ahmedhassan12@gmail.com	ahmed123@	C
15	Ahmed	Hassa	Ahmedhassan121@gmail.com	ahmed123@	C

Figure 3: Client Table

ClientID	PhoneNumber	NationalID	Address
11	01012345678	30005152345678	10 Ramses Street, Cairo, Egypt
12	01012345677	30005152345677	10 Heliplos Street, Cairo, Egypt
13	01012345666	30005152345699	10 Nasr city Street, Cairo, Egypt
14	01012345655	30005152345611	10 Nasr city Street, Cairo, Egypt
15	01012345622	30005152345612	10 Nasr city Street, Cairo, Egypt

Figure 4: Admin Table

AdminID	LastLogin
1	2026-03-19 22:41:09
3	2026-03-19 22:42:07

Figure 5: SupportTicket Table

TicketID	Issue	Resolved	AdminID	ClientID
1	Test 1	0	1	11
2	Test 2	1	1	12

Figure 6: Category Table

CategoryID	Type
1	Electronics
2	Other

Figure 7: Company Table

CompanyID	Name	ContactEmail	FoundingDate	Password	Comm	RegistrationDate	TaxNumber	VerficationStatus
1	Meta	Meta@gmail.com	2020-01-15 00:00:00	securepassword123	20.00	2024-03-17 00:00:00	123456	Verified
2	Microsoft	Microsoft@gmail.com	2000-01-01 00:00:00	Microsoft123$$$	15.34	2000-01-02 00:00:00	123444	Pending

Figure 8: CompanyEmployee Table

EmployeeID	Email	FirstName	LastName	Phone	CompanyID
1	omar@gmail.com	omar	Khaled	1671987233	1
2	Ahmed@gmail.com	Anas	Abdelrehim	1671987244	1

Figure 9: Application Table

ApplicationID	PaymentType	CompletionDate	SubmissionDate	TrackingNumber	Status ▼ 1	DeliveryAddress	CompanyEmployeeID	CategoryID
2	PARTIAL	0000-00-00 00:00:00	2026-01-15 00:00:00	A24sfkj3	Completed	Masr El gdeida	2	2
5	FULL	2026-01-20 00:00:00	2026-01-15 00:00:00	A24sfkj1	Pending	Giza	1	1

Figure 10: Document Table

DocumentID	DocType	UploadDate	VerficationStatus	ClientID	ApplicationID
2	National ID / Passport	2026-01-15 00:00:00	Accepted	11	2

Figure 11: CompanyPayment Table

CompanyPaymentID	PaymentDate	Amount	CompanyID	PaymentID
1	2026-01-20 00:00:00	15000.00	1	1
3	2026-01-20 00:00:00	15000.00	2	1

Figure 12: Payment Table

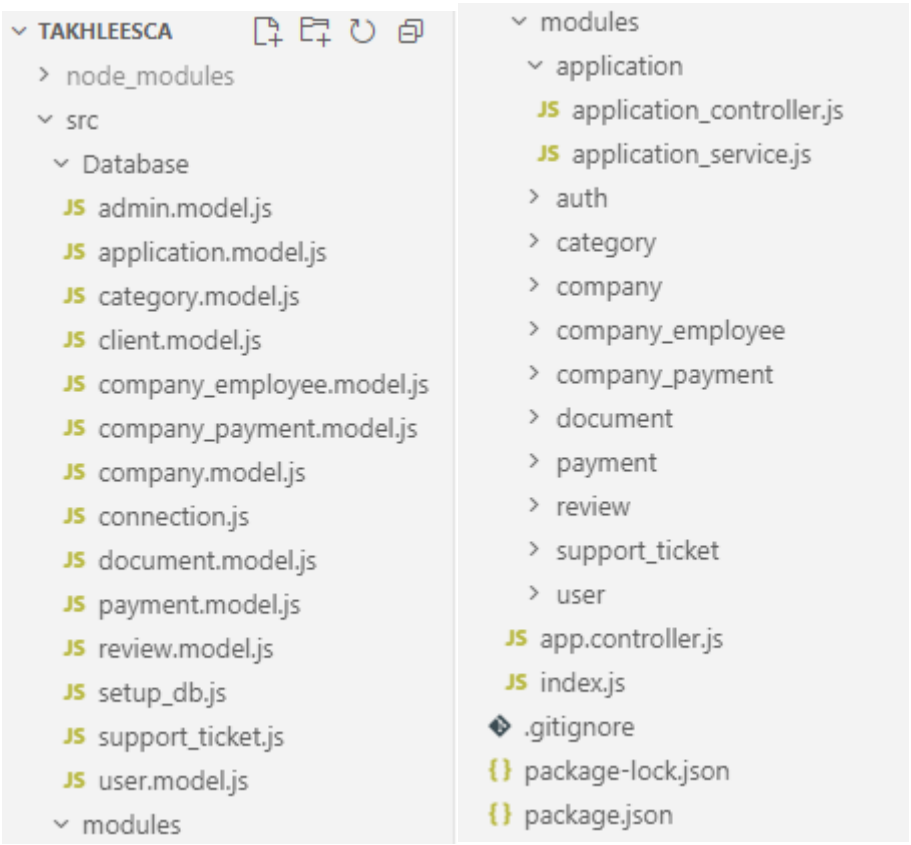
PaymentID	PaymentDate	Amount	PaymentGateway	ApplicationID
1	2026-01-20 00:00:00	30000.00	Credit Card	5

Figure 13: Review Table

ReviewID	Review	Rating	ApplicationID	CategoryID
2	Very Fast Service	4	2	1

2.2 WEB SERVER DESIGN

Figure 14: Folder Structure



Code Snapshots:

Figure 15: connection.js (imports mysql and connects to the database)

Figure 16: user.model.js (contains the function to create the User table in mysql

Figure 17: application.model.js (contains the function to create the Application table in mysql

Figure 18: company.model.js (contains the function to create the Company table in mysql)

company.model.js

src > Database > company.model.js > ...

```
1   import db from "./connection.js";
2
3   export function createCompanyTable() {
4       const sql = `
5           CREATE TABLE IF NOT EXISTS Company (
6               CompanyID INT AUTO_INCREMENT PRIMARY KEY,
7               Name VARCHAR(255) NOT NULL,
8               ContactEmail VARCHAR(255) NOT NULL,
9               FoundingDate DATETIME NOT NULL,
10              Password VARCHAR(255) NOT NULL,
11              Comm DECIMAL(4,2) NOT NULL,
12              RegistrationDate DATETIME NOT NULL,
13              TaxNumber INT NOT NULL,
14              VerficationStatus  ENUM('Pending', 'Verified', 'Rejected') NOT NULL
15          )
16      `;
17
18      return new Promise((resolve, reject) => {
19          db.query(sql, (err) => {
20              if (err) return reject(err);
21              resolve();
22          });
23      });
24  }
```

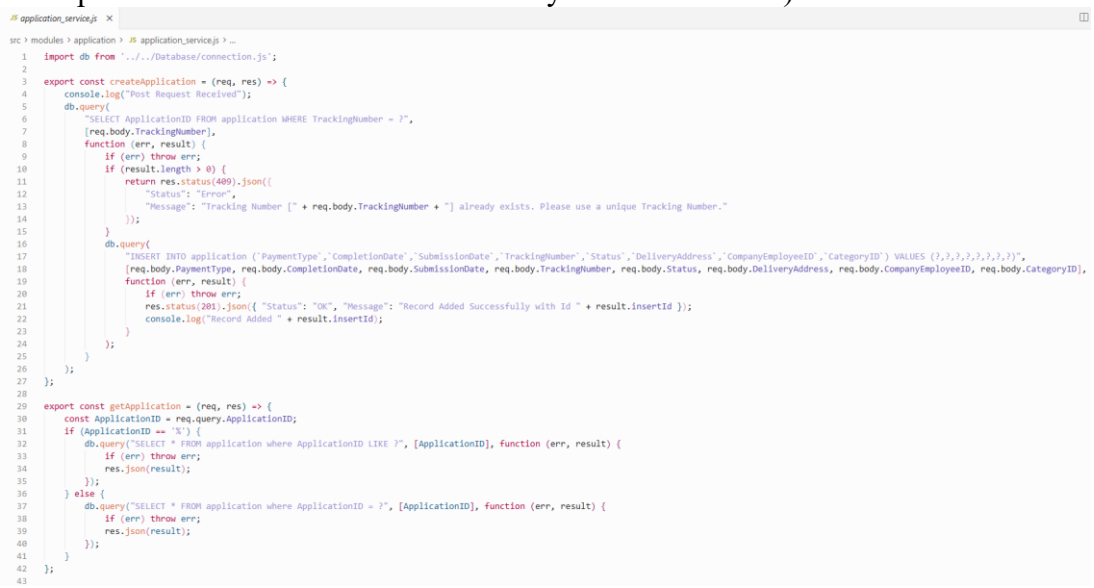
Figure 19: setup_db.js (calls all the functions to connect and create the tables of the database)

setup_db.js

src > Database > setup_db.js > ...

```
import db from './connection.js';

import { createUserTable } from './user.model.js';
import { createClientTable } from './client.model.js';
import { createAdminTable } from './admin.model.js';
import { createCompanyTable } from './company.model.js';
import { createCompanyEmployeeTable } from './company_employee.model.js';
import { createCategoryTable } from './category.model.js';
import { createApplicationTable } from './application.model.js';
import { createReviewTable } from './review.model.js';
import { createDocumentTable } from './document.model.js';
import { createPaymentTable } from './payment.model.js';
import { createCompanyPaymentTable } from './company_payment.model.js';
import { createSupportTicketTable } from './support_ticket.js';

function query(sql) {
    return new Promise((resolve, reject) => {
        db.query(sql, (err, result) => {
            if (err) return reject(err);
            resolve(result);
        });
    });
}

async function main() {
    await query('CREATE DATABASE IF NOT EXISTS Takhlees');

    await createUserTable();
    await createClientTable();
    await createAdminTable();
    await createSupportTicketTable();
    await createCompanyTable();
    await createCompanyEmployeeTable();
    await createCategoryTable();
    await createApplicationTable();
    await createReviewTable();
    await createDocumentTable();
    await createPaymentTable();
    await createCompanyPaymentTable();

    console.log('All tables are ready (created or already existed).');
    process.exit(0);
}

main().catch((err) => {
    console.error('Setup failed:', err.message);
    process.exit(1);
});
```

Figure 20: application_service.js (contains the CRUD operations as functions which are exported to be used later on. Each entity has a similar file)



```
import db from '../../Database/connection.js';

export const createApplication = (req, res) => {
    console.log("Post Request Received");
    db.query(
        "SELECT ApplicationID FROM application WHERE TrackingNumber = ?",
        [req.body.TrackingNumber],
        function (err, result) {
            if (err) throw err;
            if (result.length > 0) {
                return res.status(409).json({
                    "Status": "Error",
                    "Message": "Tracking Number [" + req.body.TrackingNumber + "] already exists. Please use a unique Tracking Number."
                });
            }
            db.query(
                "INSERT INTO application (`PaymentType`,`CompletionDate`,`SubmissionDate`,`TrackingNumber`,`Status`,`DeliveryAddress`,`CompanyEmployeeID`,`CategoryID`) VALUES (?,?,?,?,?,?,?,?)",
                [req.body.PaymentType, req.body.CompletionDate, req.body.SubmissionDate, req.body.TrackingNumber, req.body.Status, req.body.DeliveryAddress, req.body.CompanyEmployeeID, req.body.CategoryID],
                function (err, result) {
                    if (err) throw err;
                    res.status(201).json({ "Status": "OK", "Message": "Record Added Successfully with Id " + result.insertId });
                    console.log("Record Added " + result.insertId);
                }
            );
        }
    );
};

export const getApplication = (req, res) => {
    const ApplicationID = req.query.ApplicationID;
    if (ApplicationID == '%') {
        db.query("SELECT * FROM application where ApplicationID LIKE ?", [ApplicationID], function (err, result) {
            if (err) throw err;
            res.json(result);
        });
    } else {
        db.query("SELECT * FROM application where ApplicationID = ?", [ApplicationID], function (err, result) {
            if (err) throw err;
            res.json(result);
        });
    }
};
```

```
export const deleteApplication = (req, res) => {
    const ApplicationID = req.query.ApplicationID;
    db.query("SELECT ApplicationID FROM application WHERE ApplicationID = ?", [ApplicationID], function (err, result) {
        if (err) throw err;
        if (result.length === 0) {
            return res.status(404).json({
                "Status": "Error",
                "Message": "Record Id [" + ApplicationID + "] does not exist or has already been deleted."
            });
        }
        db.query("DELETE FROM application WHERE ApplicationID = ?", [ApplicationID], function (err, result) {
            if (err) throw err;
            res.status(200).json({ "Status": "OK", "Message": "Record Id [" + ApplicationID + "] deleted Successfully" });
            console.log("Delete Request Received for record [" + ApplicationID + "] received");
        });
    });
};

export const updateApplication = (req, res) => {
    console.log("PUT Request Received");
    const ApplicationID = req.query.ApplicationID;
    db.query("UPDATE application SET `Status`= ? WHERE ApplicationID = " + ApplicationID,
        [req.body.Status], function (err, result) {
            if (err) throw err;
            res.json({ "Status": "OK", "Message": "Record Id [" + ApplicationID + "] is Updated Successfully" });
            console.log("Record Id [" + ApplicationID + "] is Updated Successfully");
        });
};
```

Figure 21: application_controller.js (contains routers that call the CRUD functions from the service file to be used in REST APIs then exports these routers)

application_controller.js

src > modules > application > application_controller.js > default

```
1   import { Router } from "express";
2   import * as applicationService from "./application_service.js";
3
4   const router = Router();
5
6   router.post("/", applicationService.createApplication);
7   router.get("/", applicationService.getApplication);
8   router.delete("/", applicationService.deleteApplication);
9   router.put("/", applicationService.updateApplication);
10
11  export default router;
```

Figure 22: review_service.js

review_service.js

src > modules > review > review_service.js > ...

```
1   import db from '../../Database/connection.js';
2
3   export const createReview = (req, res) => {
4       console.log("Post Request Received");
5       db.query("INSERT INTO review (`Review`,`Rating`,`ApplicationID`,`CategoryID`) VALUES (?,?,?,?)",
6           [req.body.Review, req.body.Rating, req.body.ApplicationID, req.body.CategoryID], function (err, result) {
7               if (err) throw err;
8               res.json({ "Status": "OK", "Message": "Record Added Successfully with Id " + result.insertId });
9               console.log("Record Added" + result.insertId);
10          });
11  };
12
13  export const getReview = (req, res) => {
14      const ReviewID = req.query.ReviewID;
15      if (ReviewID == '%') {
16          db.query("SELECT * FROM review where ReviewID LIKE ?", [ReviewID], function (err, result) {
17              if (err) throw err;
18              res.json(result);
19          });
20      } else {
21          db.query("SELECT * FROM review where ReviewID = ?", [ReviewID], function (err, result) {
22              if (err) throw err;
23              res.json(result);
24          });
25      }
26  };
27
28  export const deleteReview = (req, res) => {
29      const ReviewID = req.query.ReviewID;
30      db.query("DELETE FROM review where ReviewID = ?", [ReviewID], function (err, result) {
31          if (err) throw err;
32          res.json({ "Status": "OK", "Message": "Record Id [" + ReviewID + "] deleted Successfully" });
33          console.log("Delete Request Received for record [" + ReviewID + "] received");
34      });
35  };
36
37  export const updateReview = (req, res) => {
38      console.log("PUT Request Received");
39      const ReviewID = req.query.ReviewID;
40      db.query("UPDATE review SET `Rating`= ? WHERE ReviewID = " + ReviewID,
41          [req.body.Rating], function (err, result) {
42              if (err) throw err;
43              res.json({ "Status": "OK", "Message": "Record Id [" + ReviewID + "] is Updated Successfully" });
44              console.log("Record Id [" + ReviewID + "] is Updated Successfully");
45          });
46  };
```

Figure 23: review_controller.js

review_controller.js

src > modules > review > review_controller.js > default

```
1   import { Router } from "express";
2   import * as reviewService from "./review_service.js";
3
4   const router = Router();
5
6   router.post("/", reviewService.createReview);
7   router.get("/", reviewService.getReview);
8   router.delete("/", reviewService.deleteReview);
9   router.put("/", reviewService.updateReview);
10
11  export default router;
```

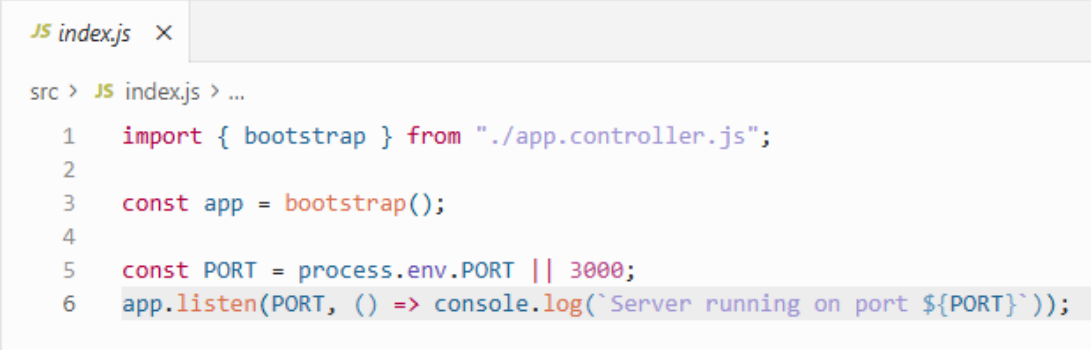
Figure 24: app.controller.js (imports all the routers from all the controller files and manages the URLs)

app.controller.js

src > app.controller.js > ...

```
import express from "express";

import authRouter from "./modules/auth/auth.controller.js";
import userRouter from "./modules/user/user_controller.js";
import applicationRouter from "./modules/application/application_controller.js";
import categoryRouter from "./modules/category/category_controller.js";
import companyRouter from "./modules/company/company_controller.js";
import companyEmployeeRouter from "./modules/company_employee/company_employee_controller.js";
import companyPaymentRouter from "./modules/company_payment/company_payment_controller.js";
import documentRouter from "./modules/document/document_controller.js";
import paymentRouter from "./modules/payment/payment_controller.js";
import reviewRouter from "./modules/review/review_controller.js";
import supportTicketRouter from "./modules/support_ticket/support_ticket_controller.js";

export const bootstrap = () => {
  const app = express();

  // ------------------------------
  // Middlewares
  // ------------------------------
  app.use(express.json());

  // ------------------------------
  // Main Router (API Mount)
  // ------------------------------
  app.use("/auth", authRouter);
  app.use("/user", userRouter);
  app.use("/application", applicationRouter);
  app.use("/category", categoryRouter);
  app.use("/company", companyRouter);
  app.use("/companyemployee", companyEmployeeRouter);
  app.use("/companypayment", companyPaymentRouter);
  app.use("/document", documentRouter);
  app.use("/payment", paymentRouter);
  app.use("/review", reviewRouter);
  app.use("/supportticket", supportTicketRouter);

  // ------------------------------
  // 404 Handler
  // ------------------------------
  app.use((req, res) => {
    res.status(404).json({ ok: false, message: "Route not found" });
  });

  // ------------------------------
  // Error Handler
  // ------------------------------
  app.use((err, req, res, next) => {
    console.error(err);
    res.status(err.status || 500).json({
```

Figure 25: index.js (imports the app.controller.js routers and sets the port to listen on)



2.3 API DESIGN

Postman testing results:

Figure 26: Get method for User entity

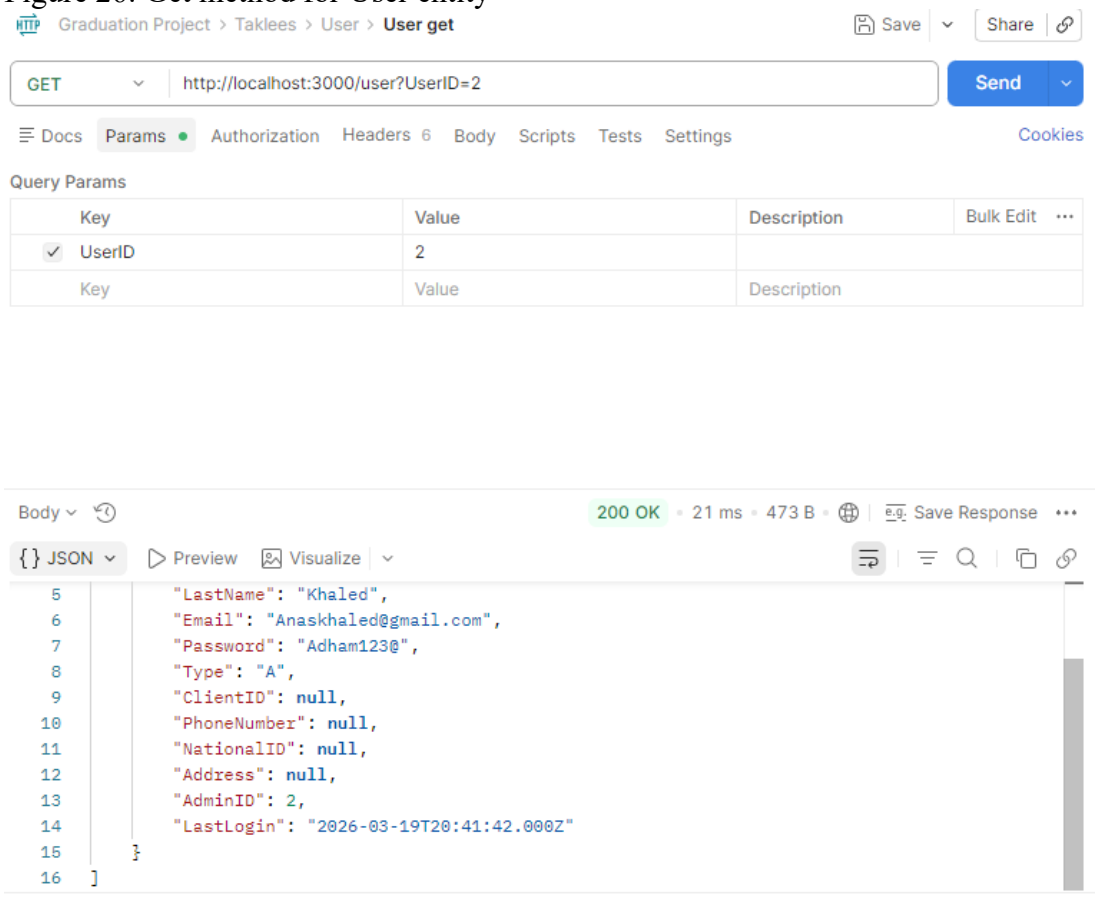


Figure 27: Post method for User entity

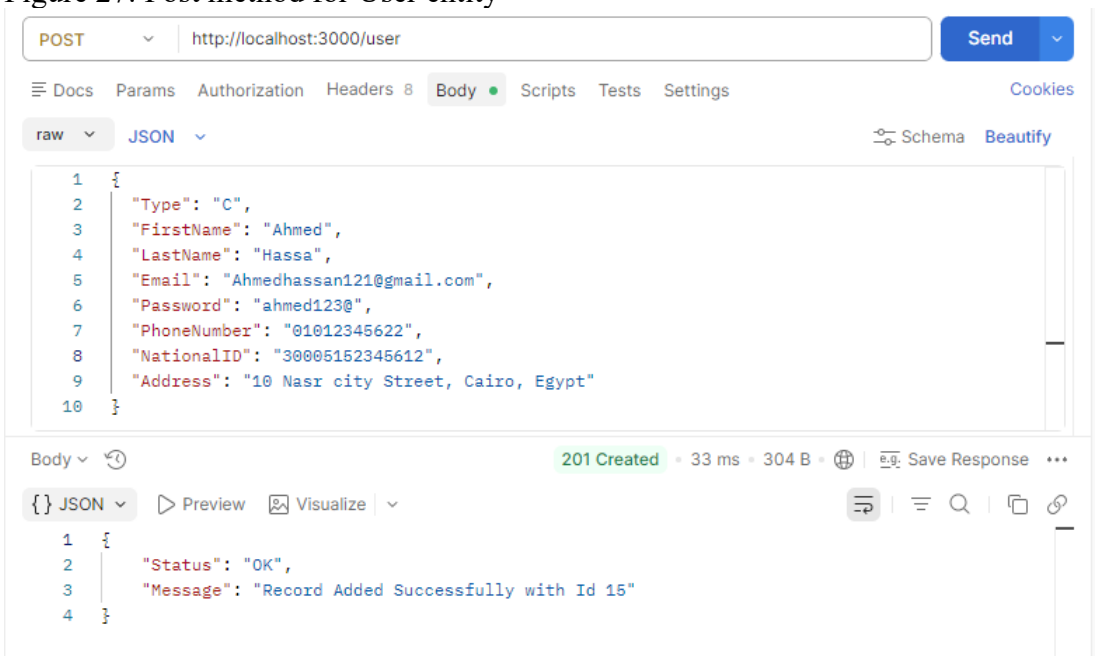


Figure 28: DELETE method for User entity

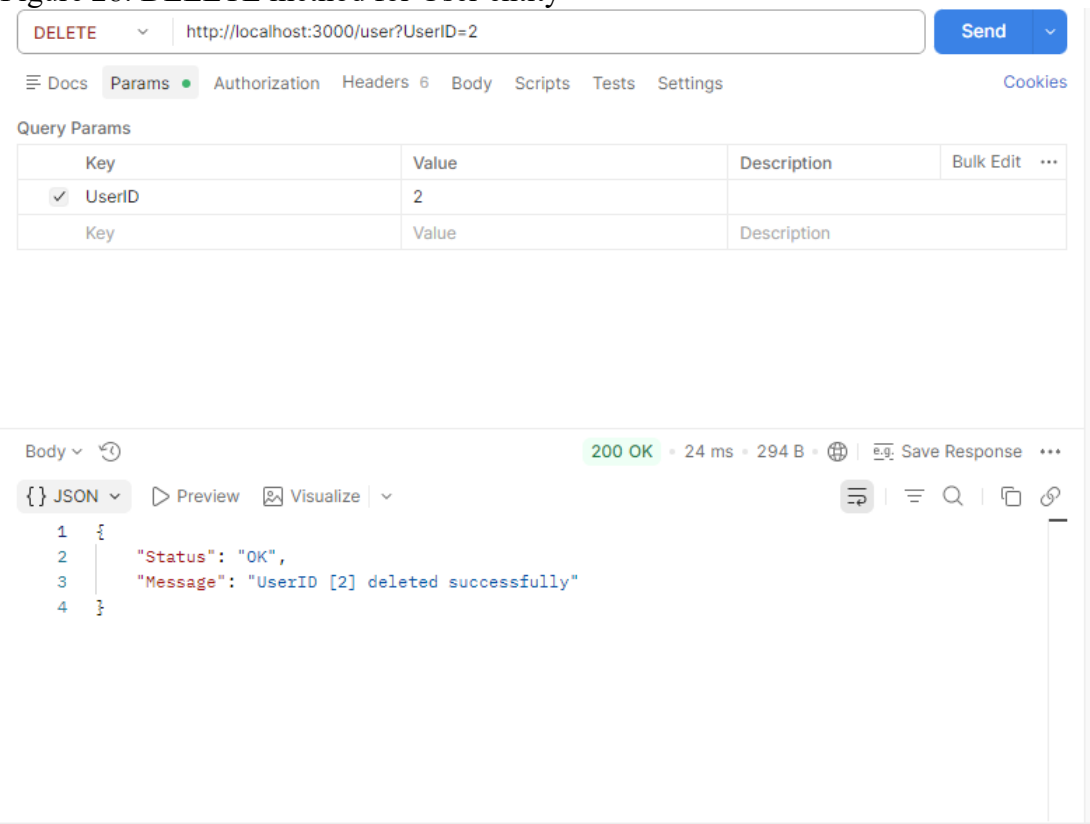


Figure 29: Put method for User entity

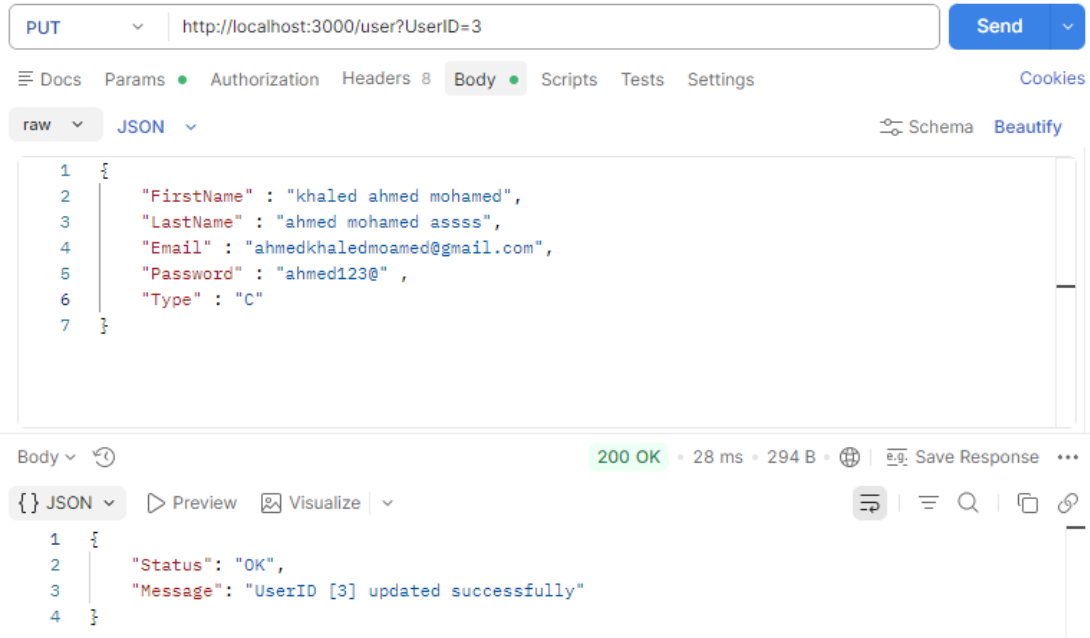


Figure 30: DELETE method for CompanyPayment entity

Figure 31: GET method for CompanyPayment entity

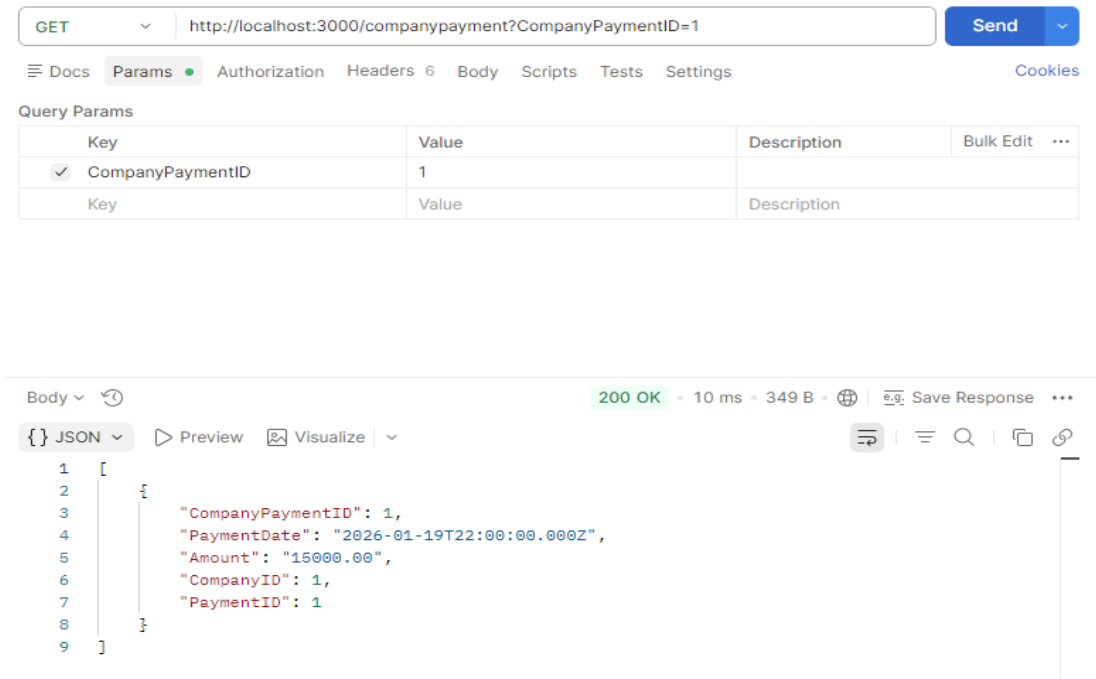


Figure 32: POST method for CompanyPayment entity

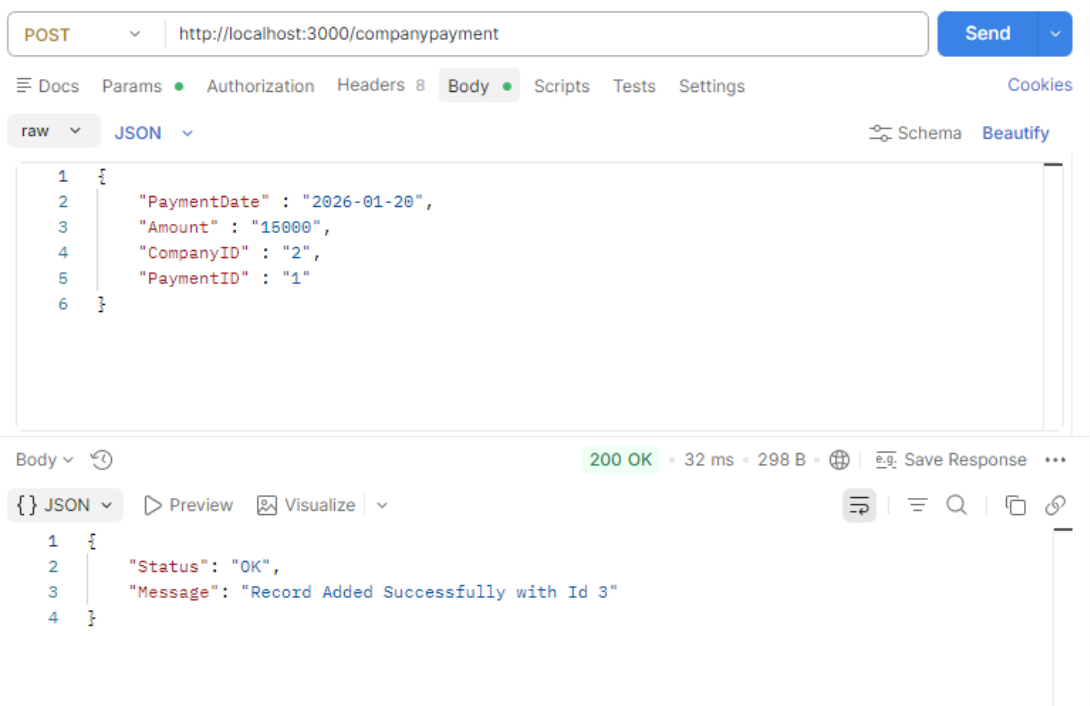


Figure 33: PUT method for CompanyPayment entity

Figure 34: GET method for Document entity

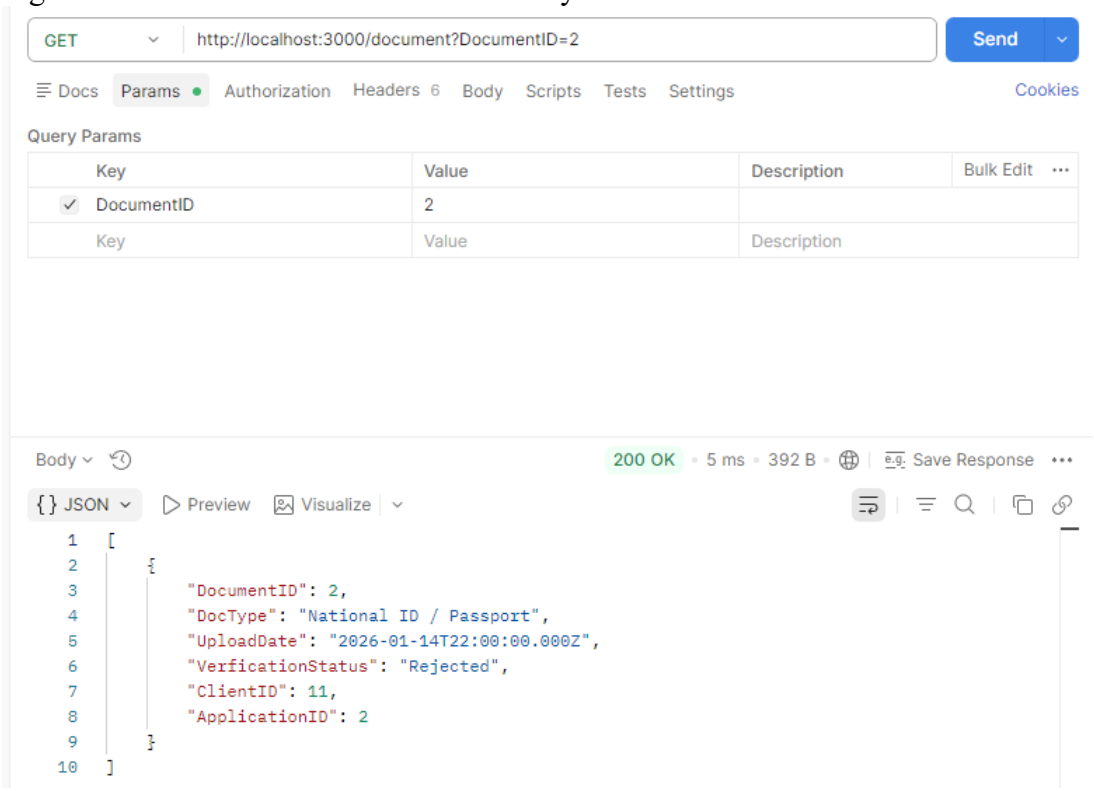


Figure 35: DELETE method for Document entity

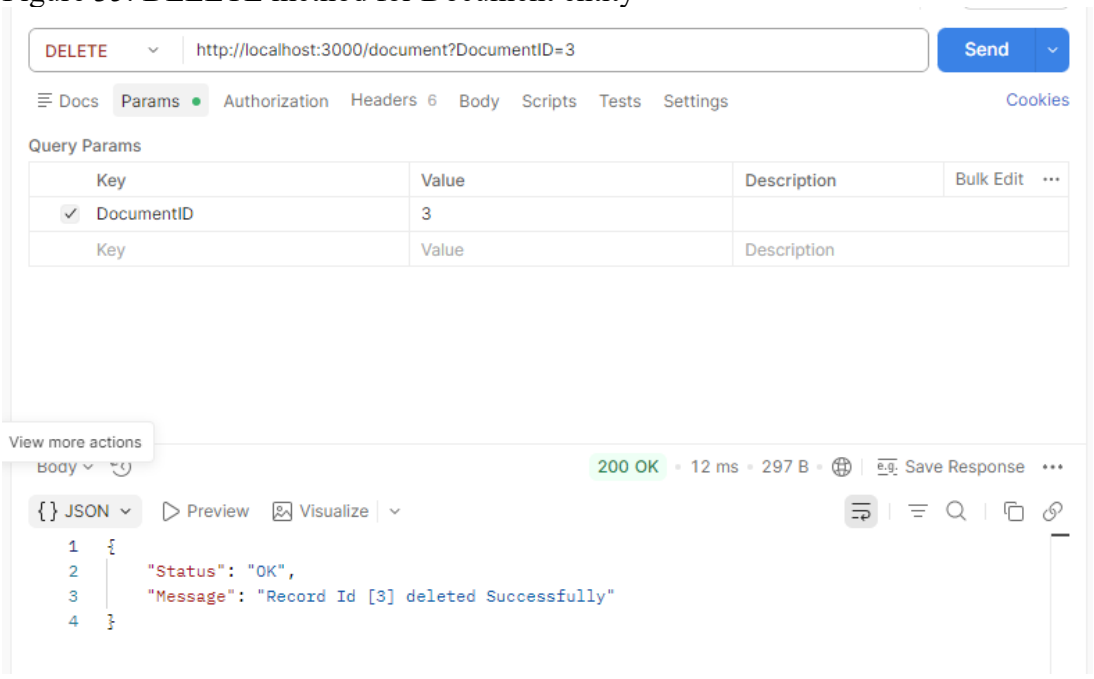


Figure 36: POST method for Document entity

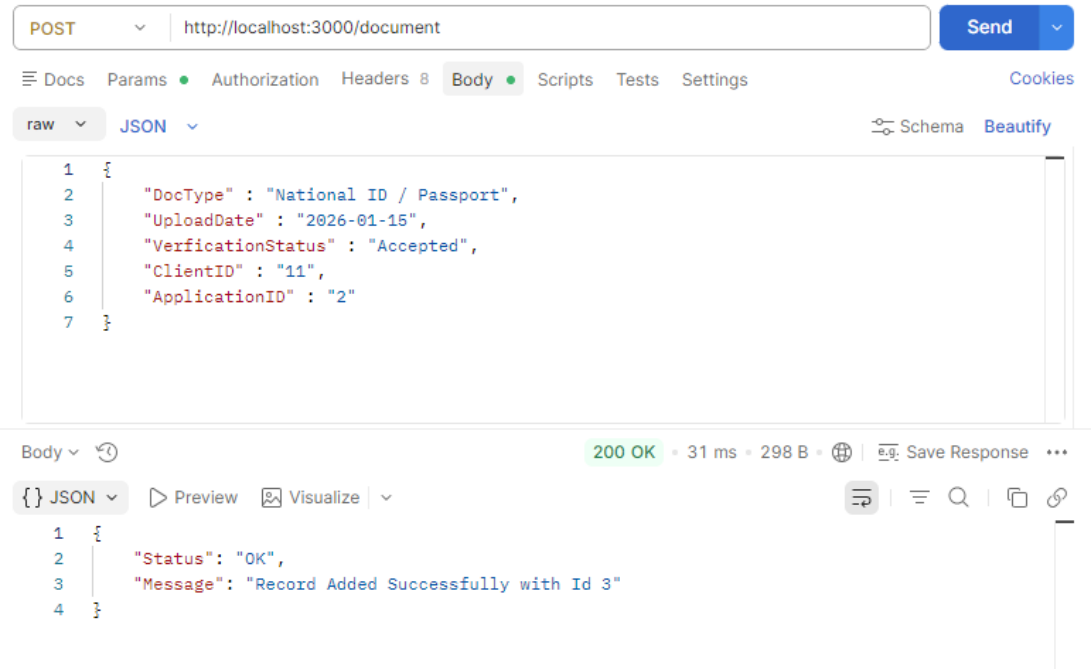


Figure 37: PUT method for Document entity

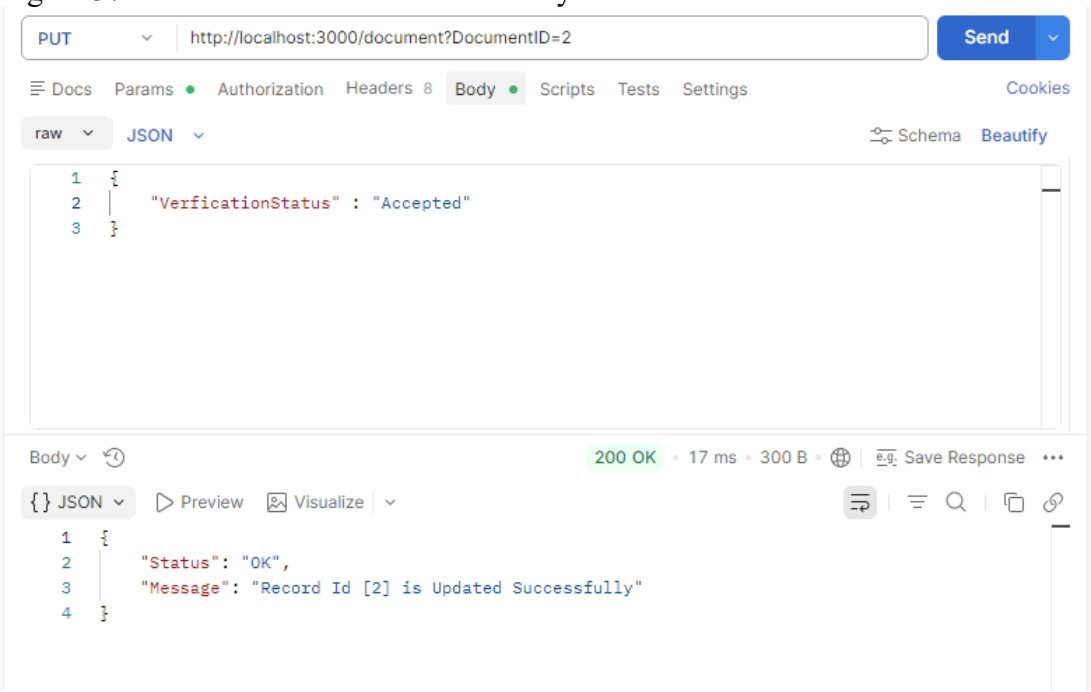


Figure 38: DELETE method for Payment entity

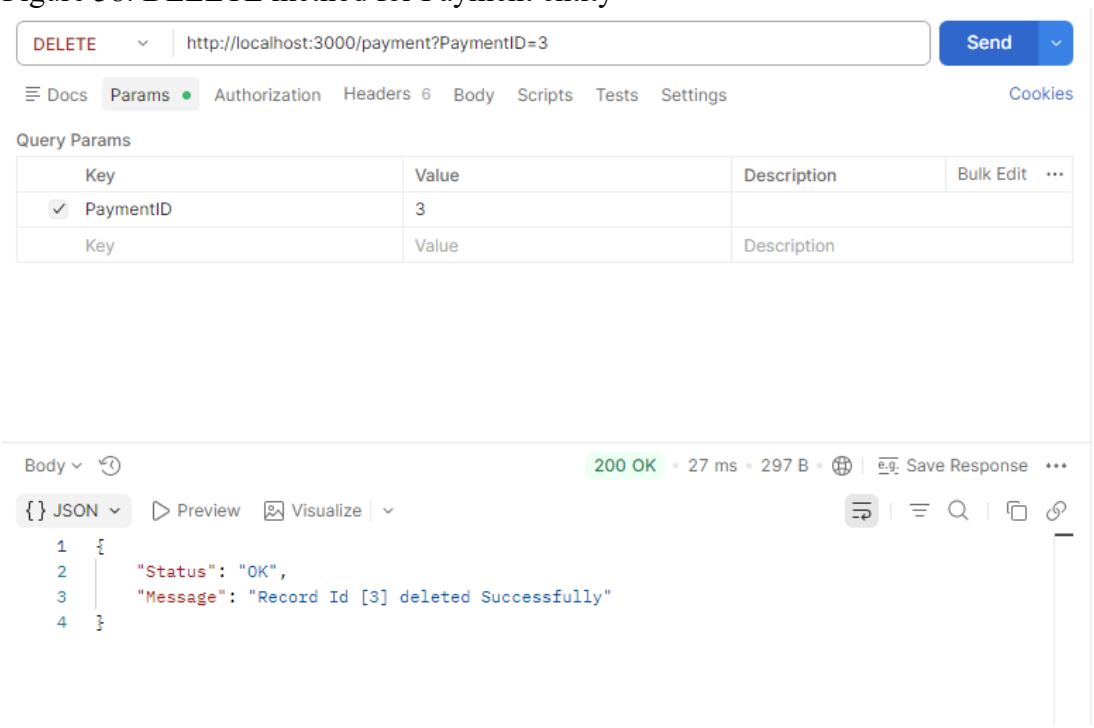


Figure 39: GET method for Payment entity

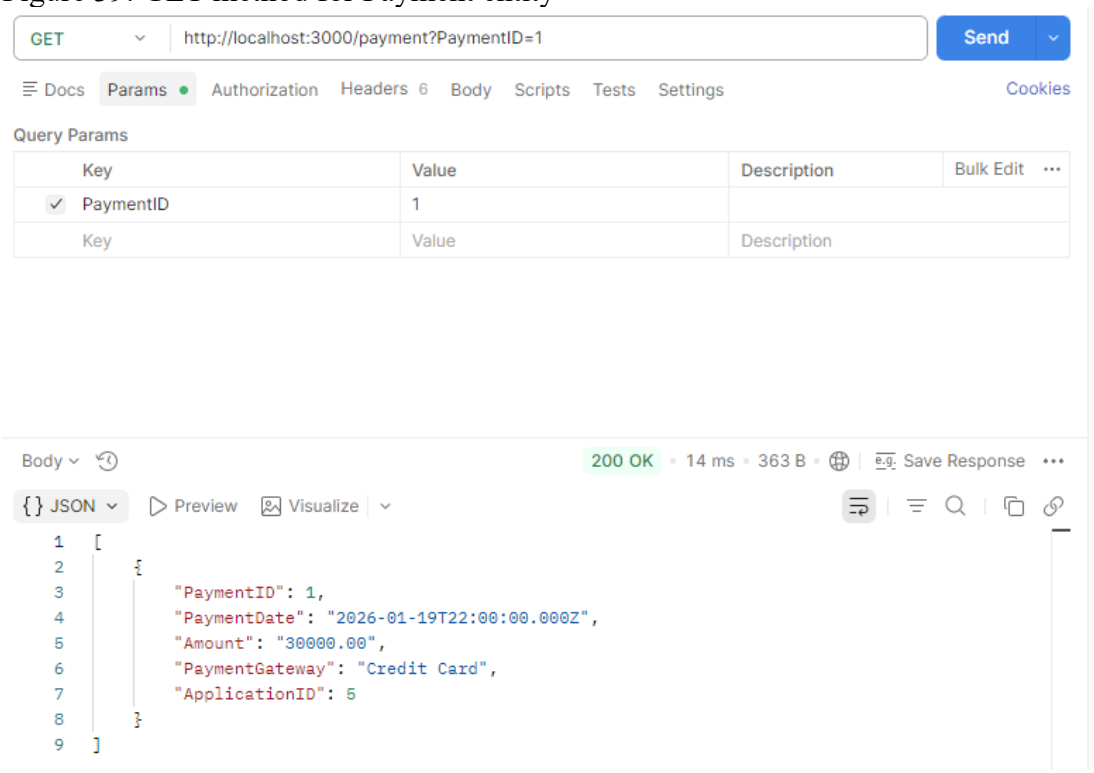


Figure 40: POST method for Payment entity

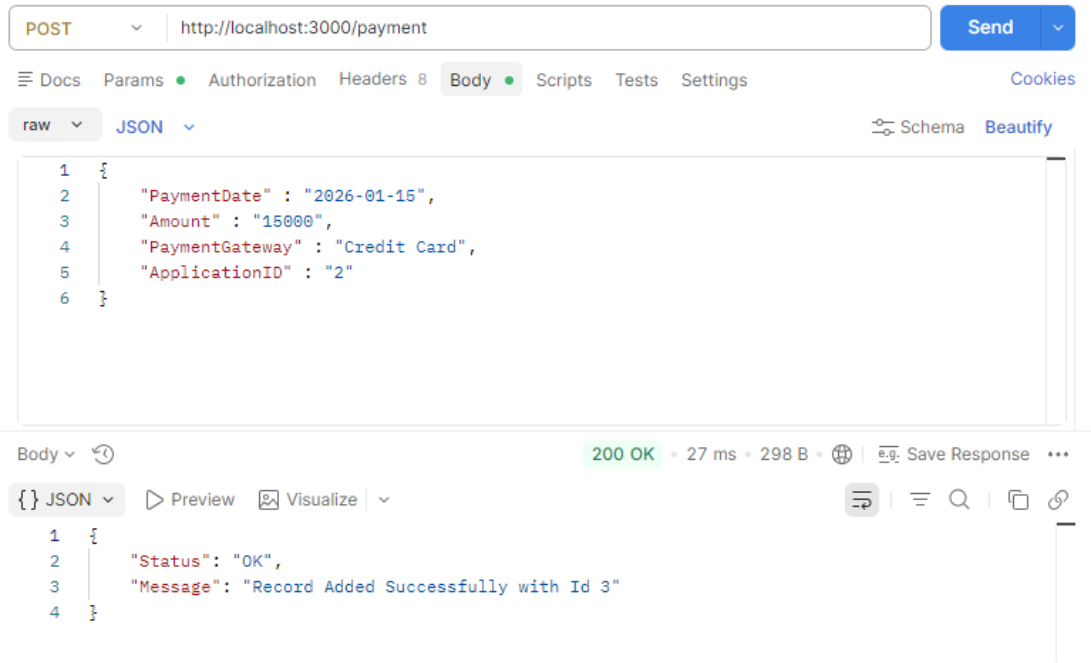


Figure 41: PUT method for Payment entity

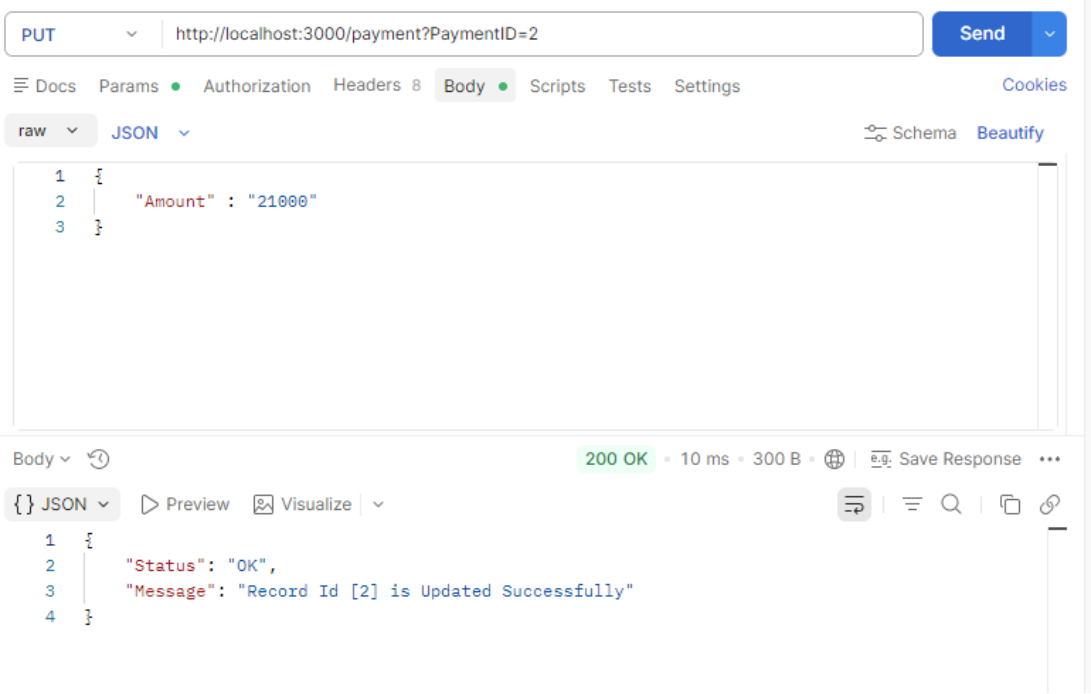


Figure 42: DELETE method for Application entity

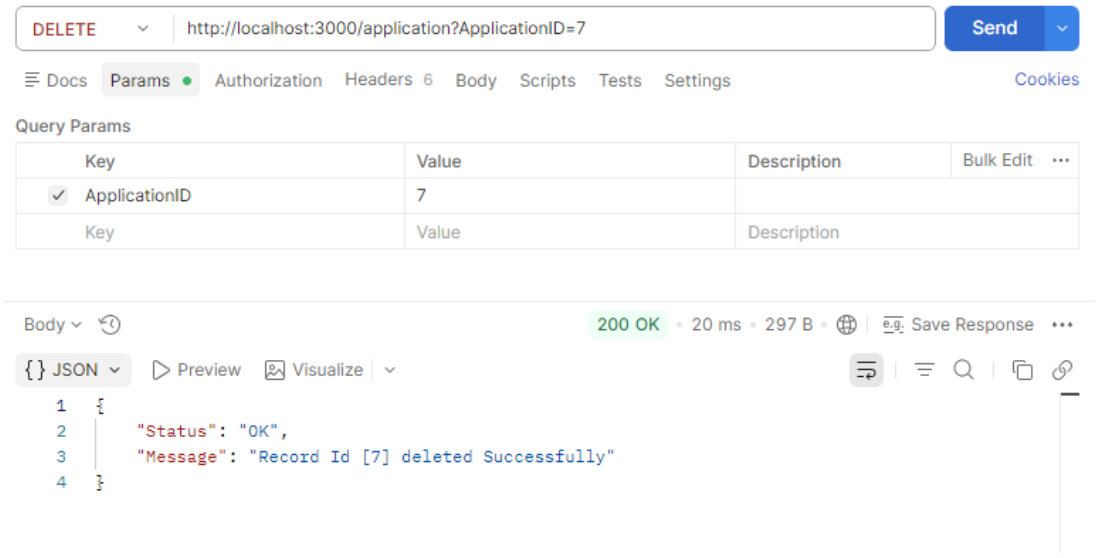


Figure 43: GET method for Application entity

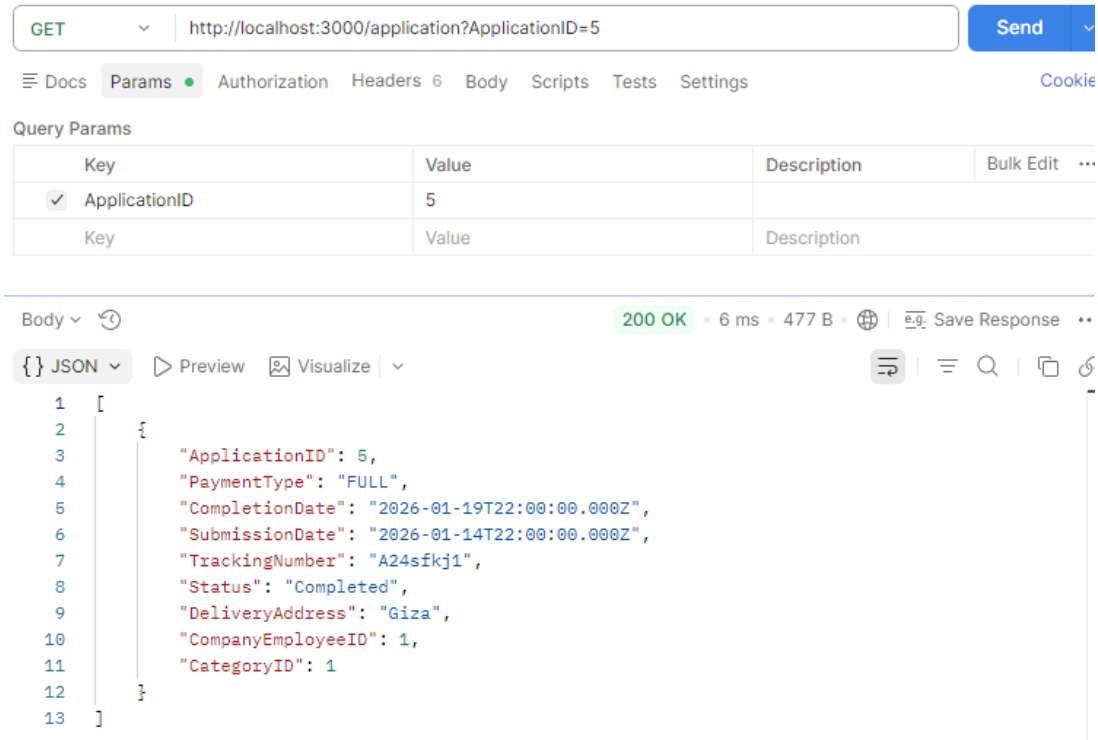


Figure 44: POST method for Application entity

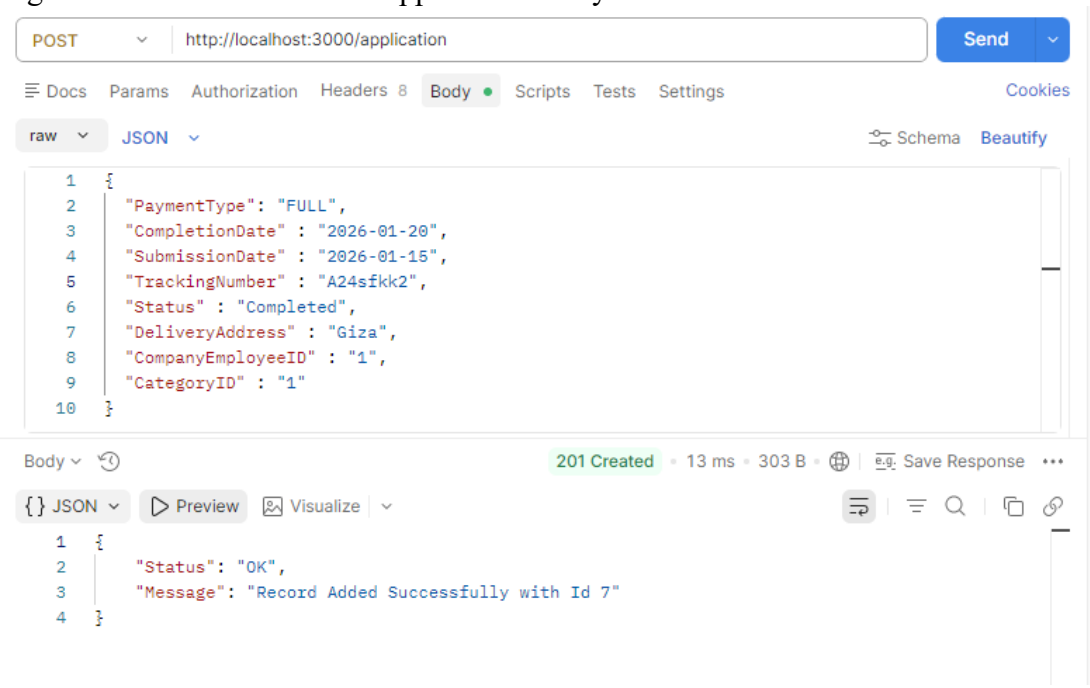


Figure 45: PUT method for Application entity

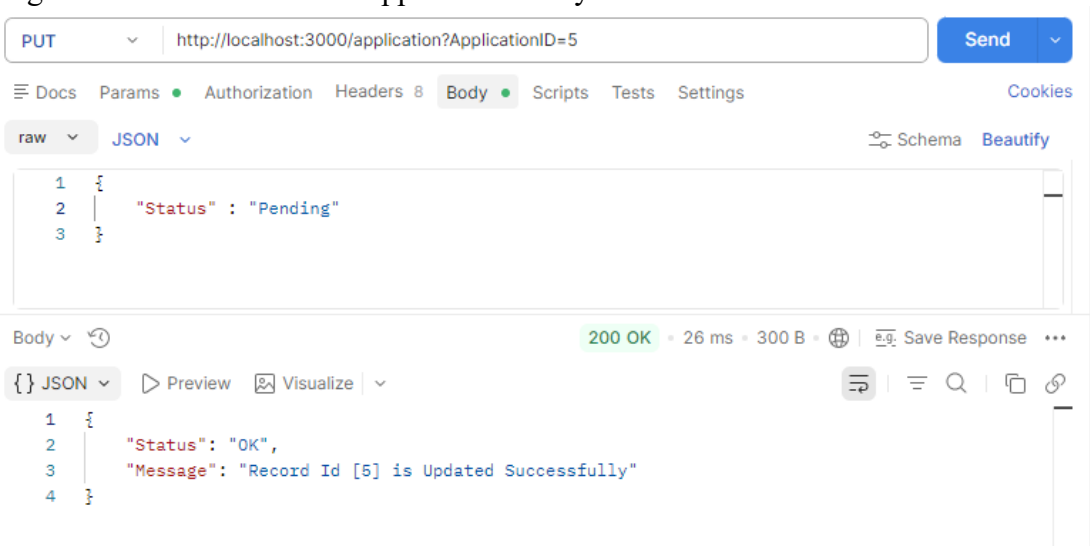


Figure 46: GET method for SupportTicket entity

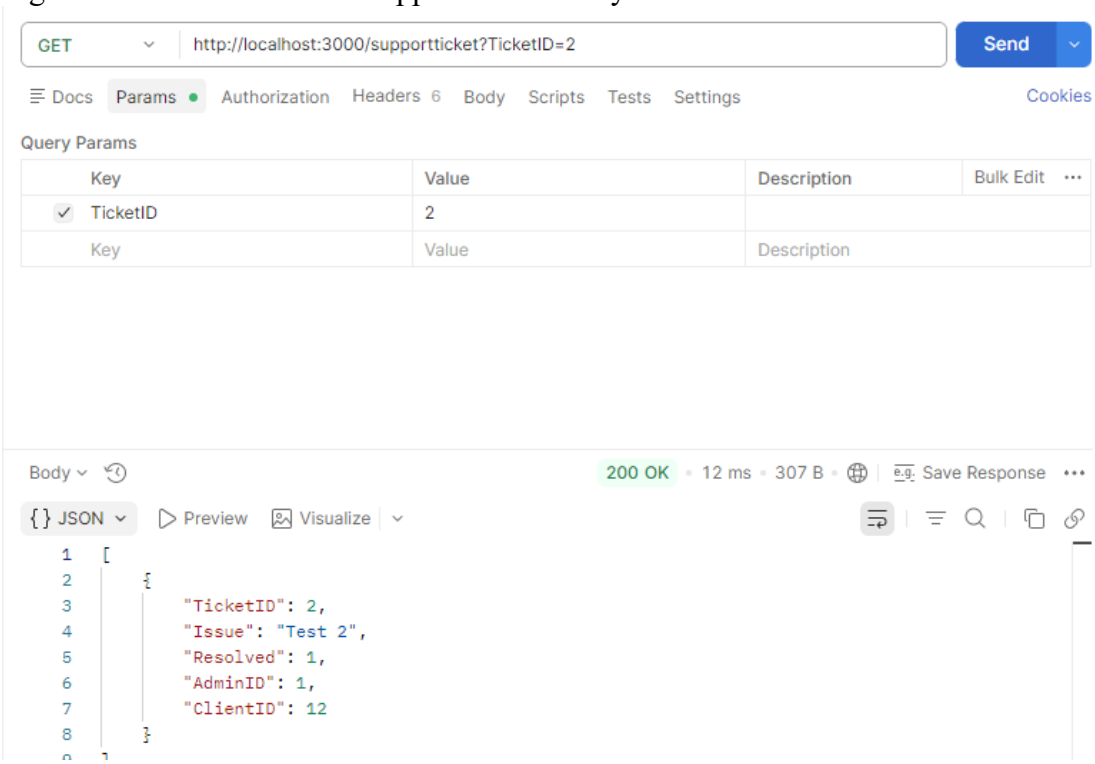


Figure 47: PUT method for SupportTicket entity

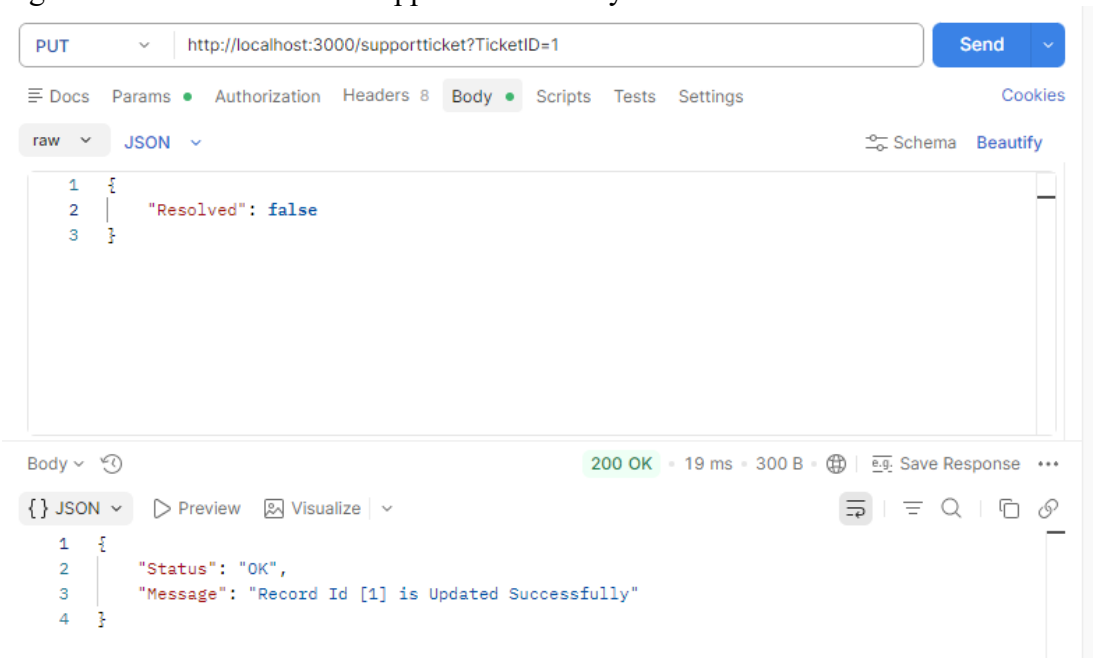


Figure 48: DELETE method for SupportTicket entity

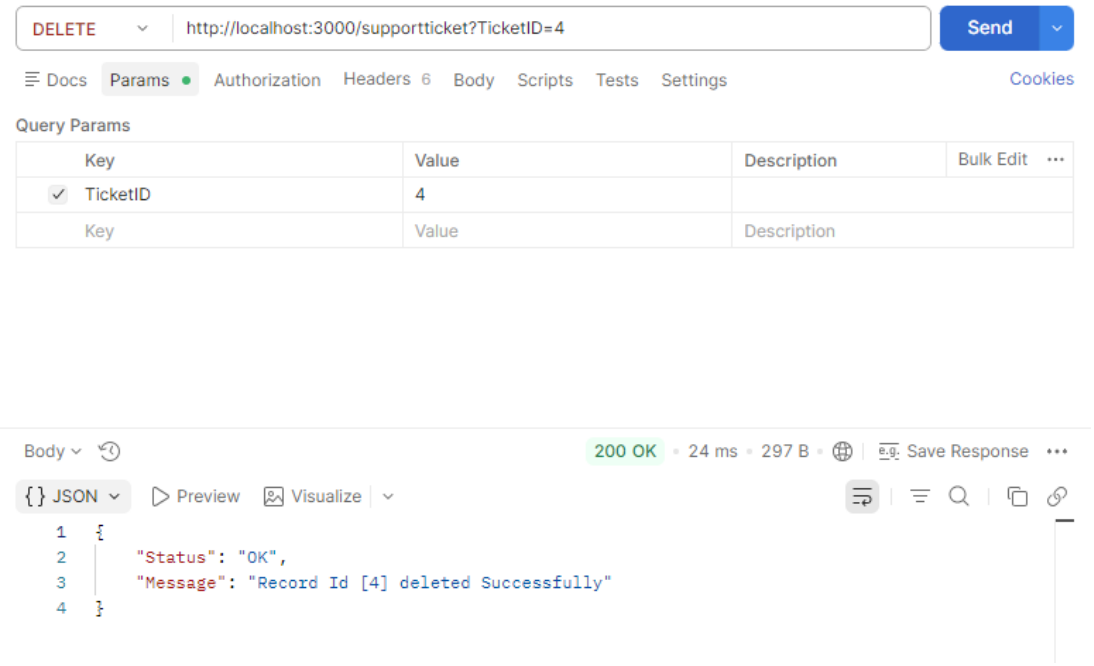


Figure 49: POST method for SupportTicket entity

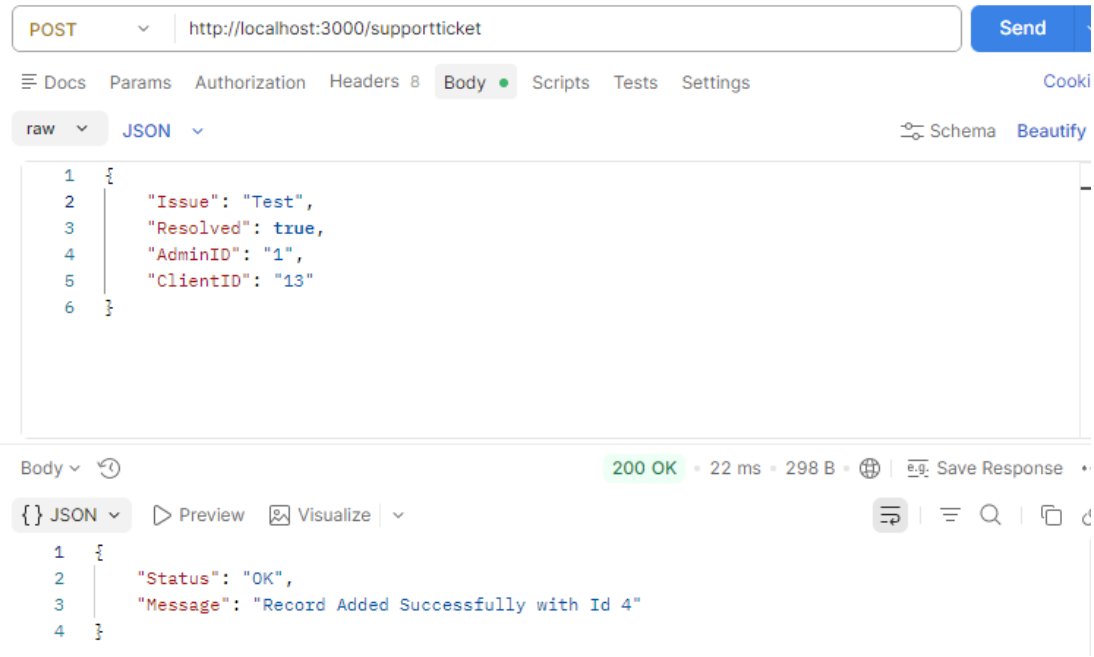


Figure 50: PUT method for Review entity

Figure 51: POST method for Review entity

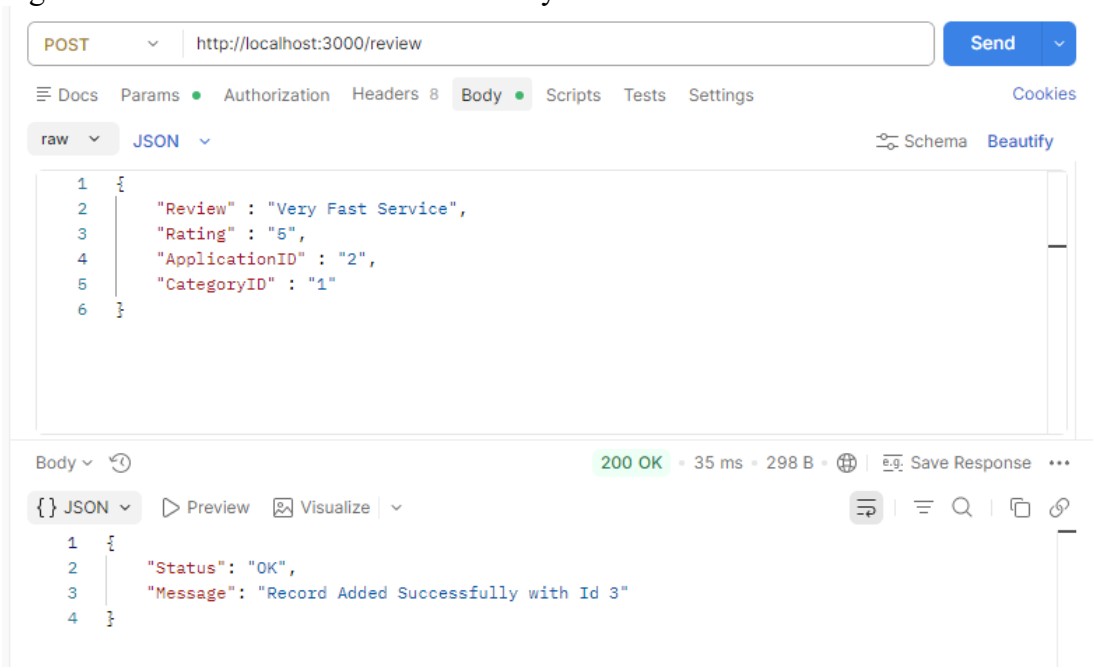


Figure 52: DELETE method for Review entity

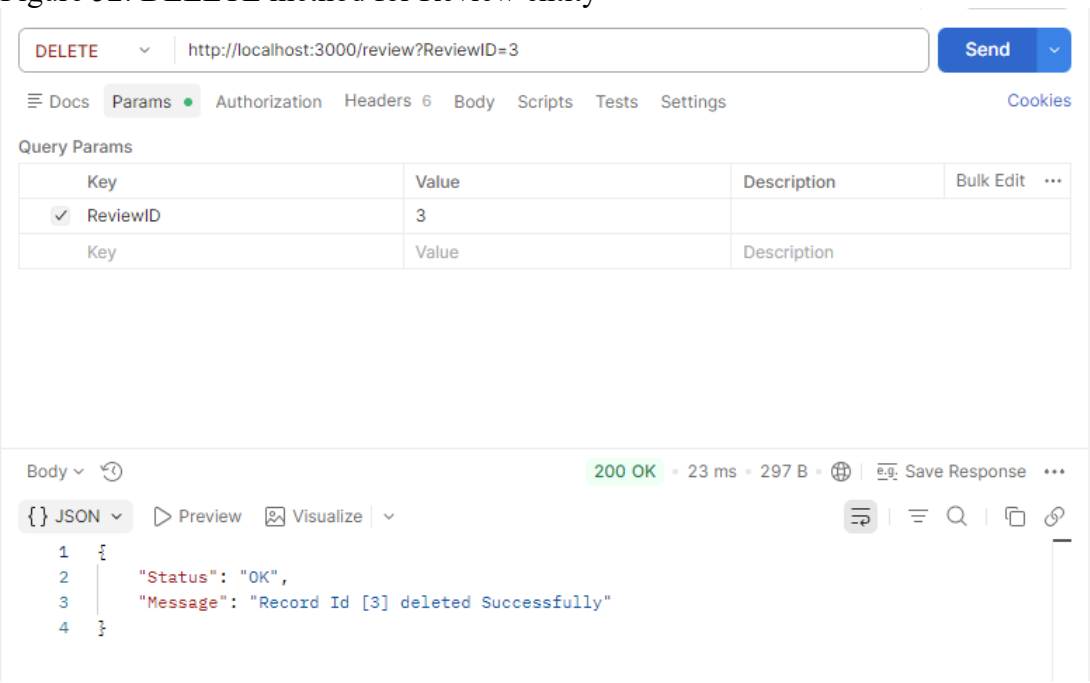


Figure 53: GET method for Review entity

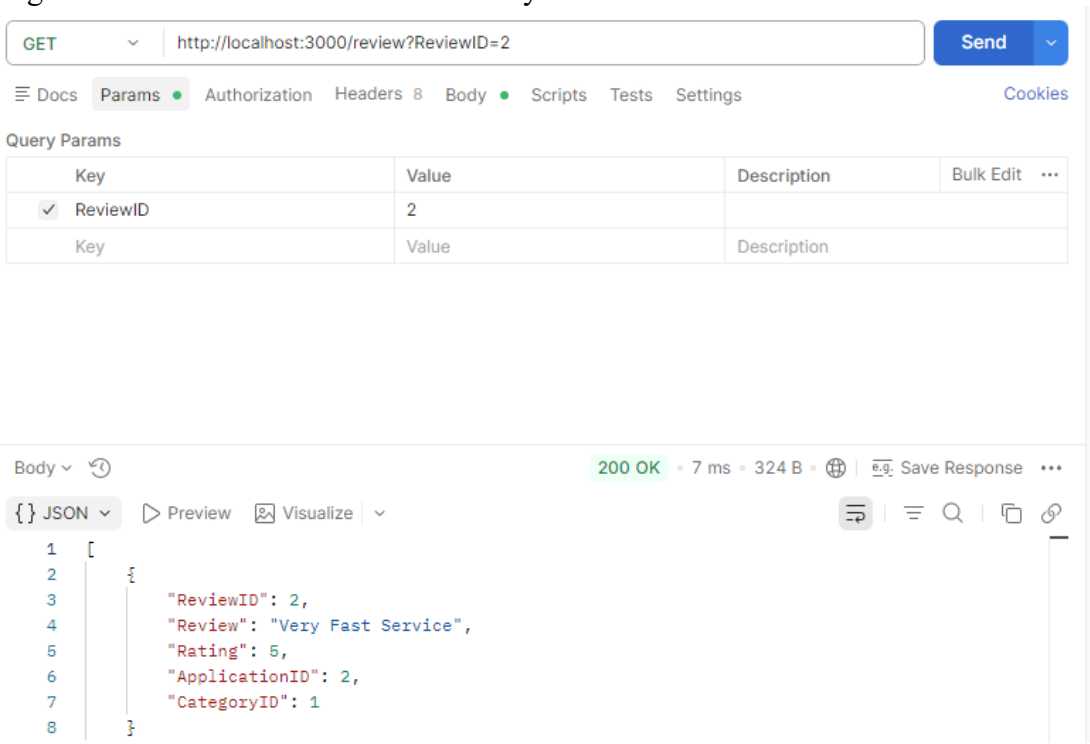


Figure 54: PUT method for Category entity

Figure 55: POST method for Category entity

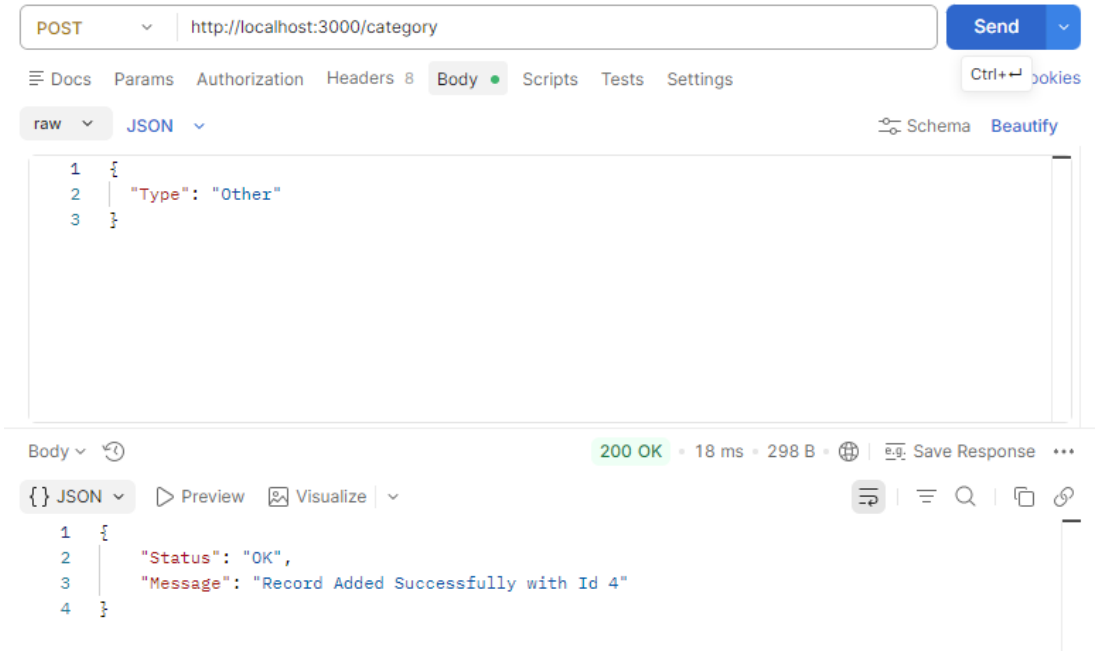


Figure 56: GET method for Category entity

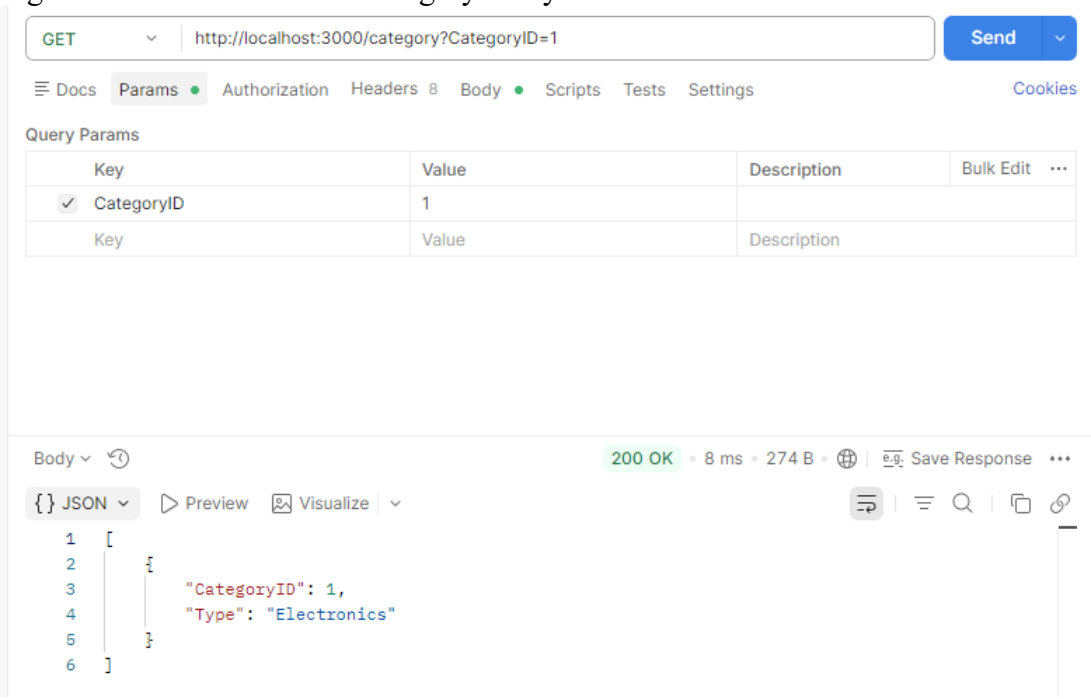


Figure 57: DELETE method for Category entity

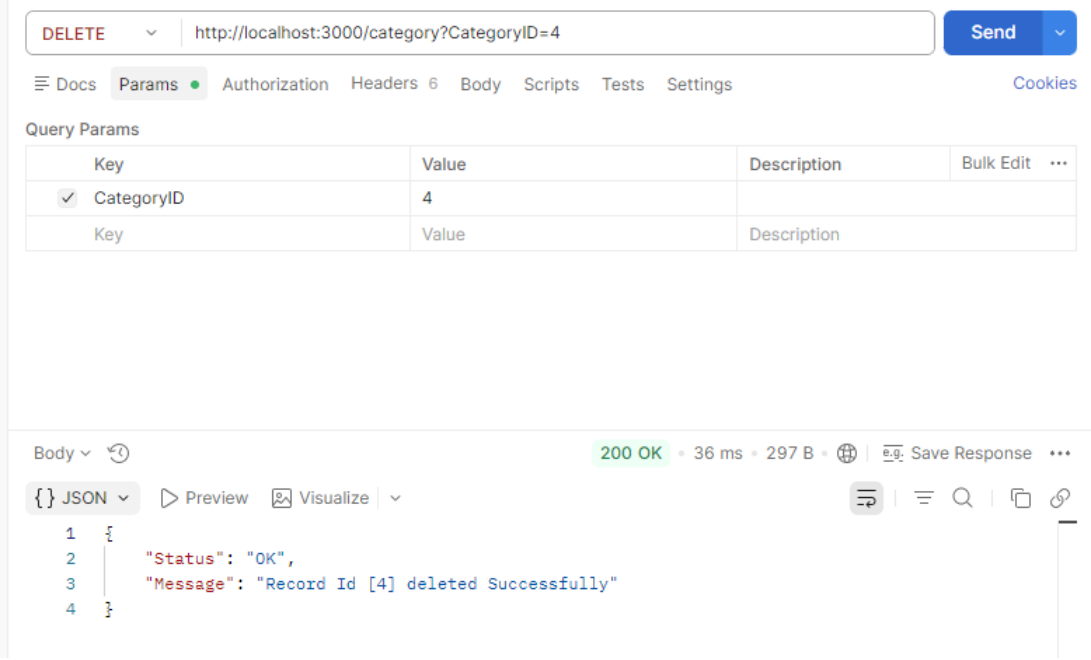


Figure 58: DELETE method for CompanyEmployee entity

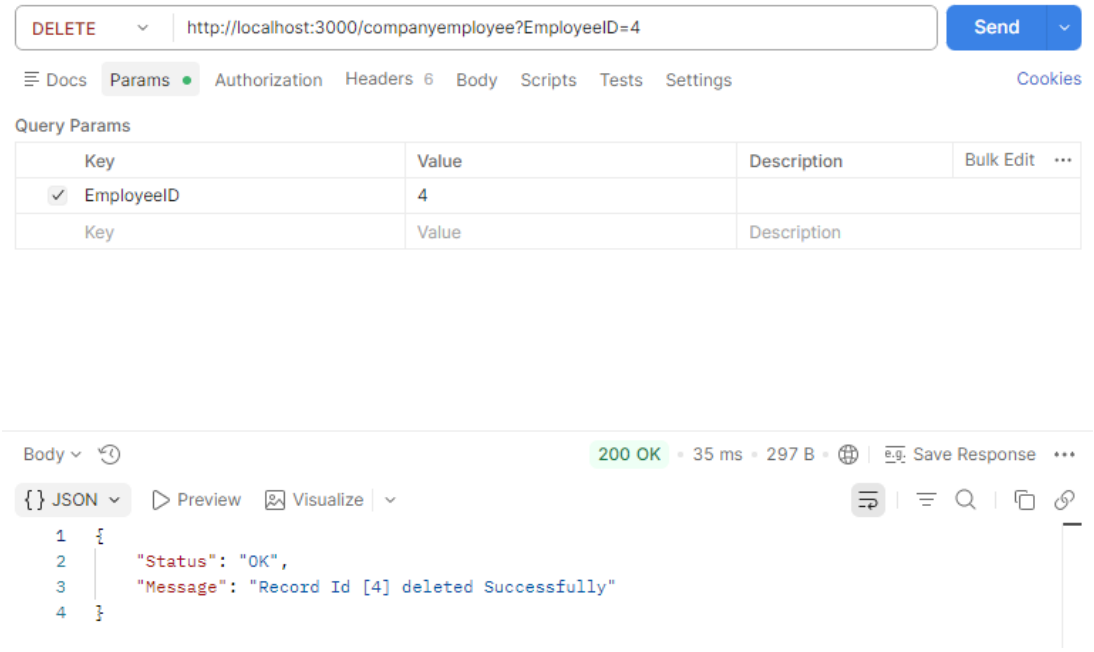


Figure 59: GET method for CompanyEmployee entity

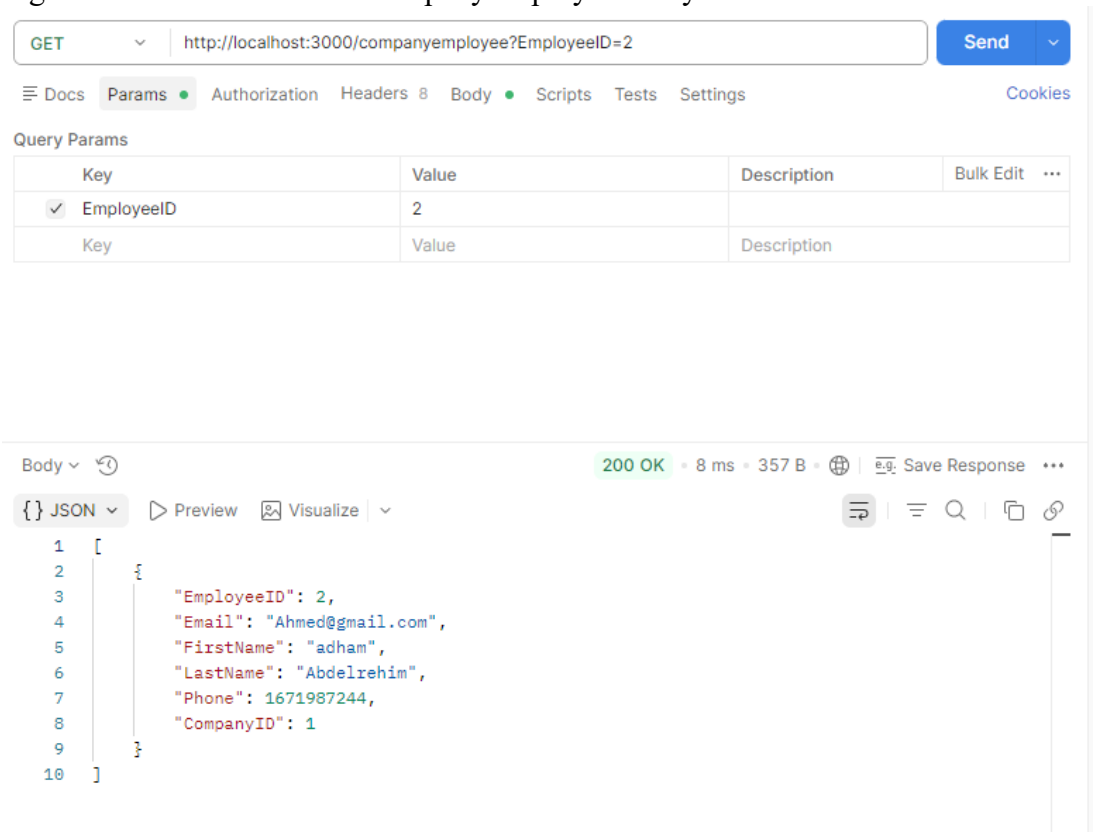


Figure 60: POST method for CompanyEmployee entity

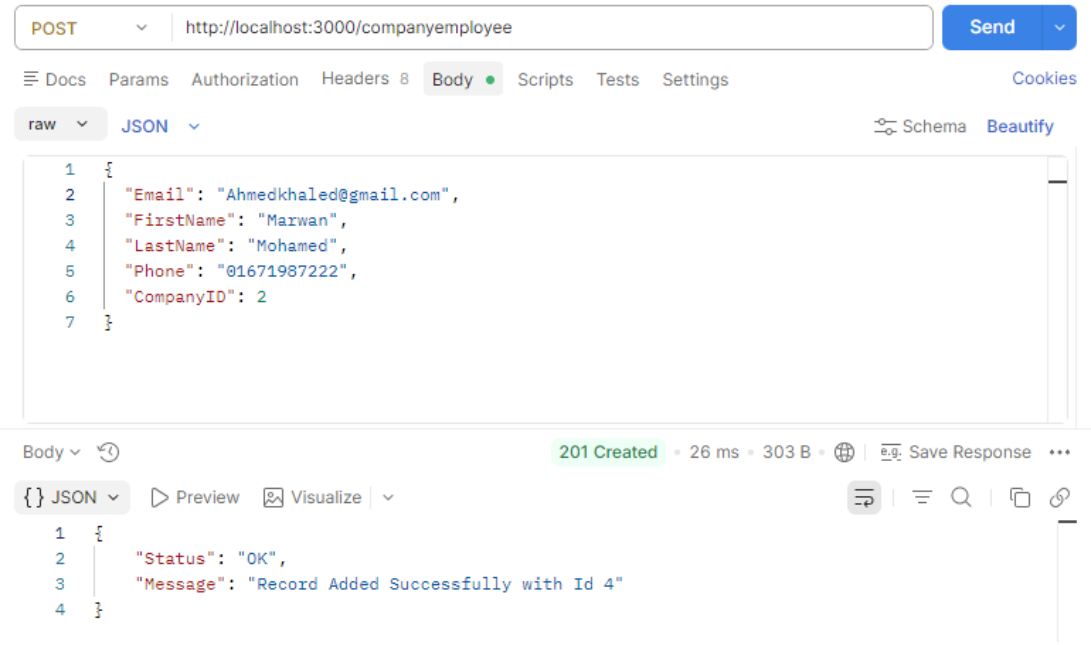


Figure 61: PUT method for CompanyEmployee entity

Figure 62: PUT method for Company entity

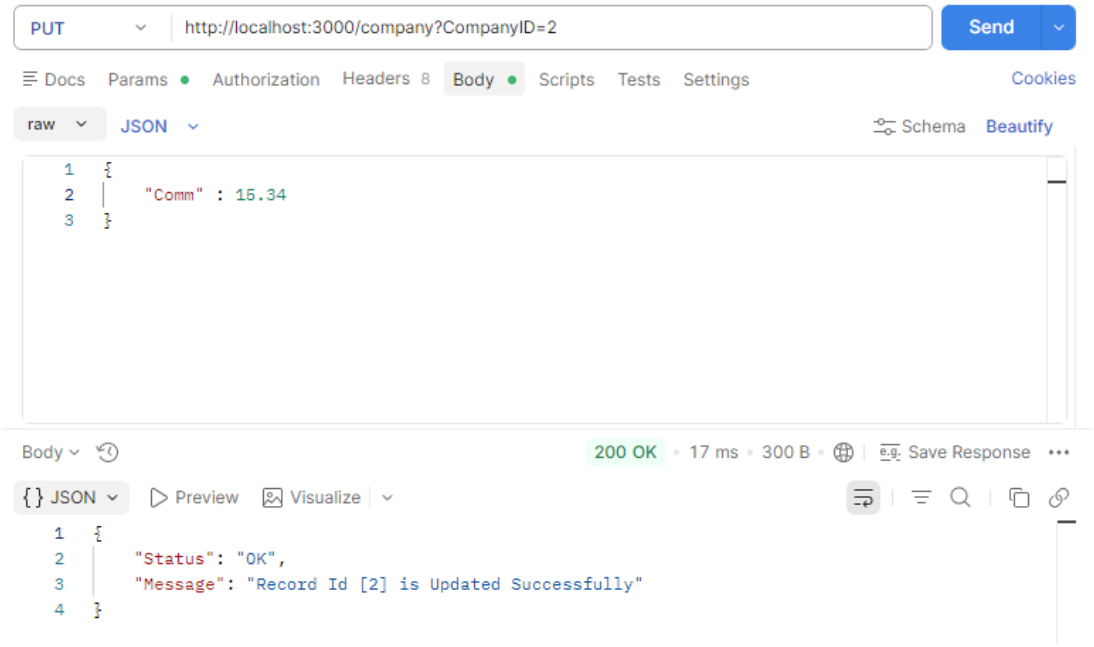


Figure 63: DELETE method for Company entity

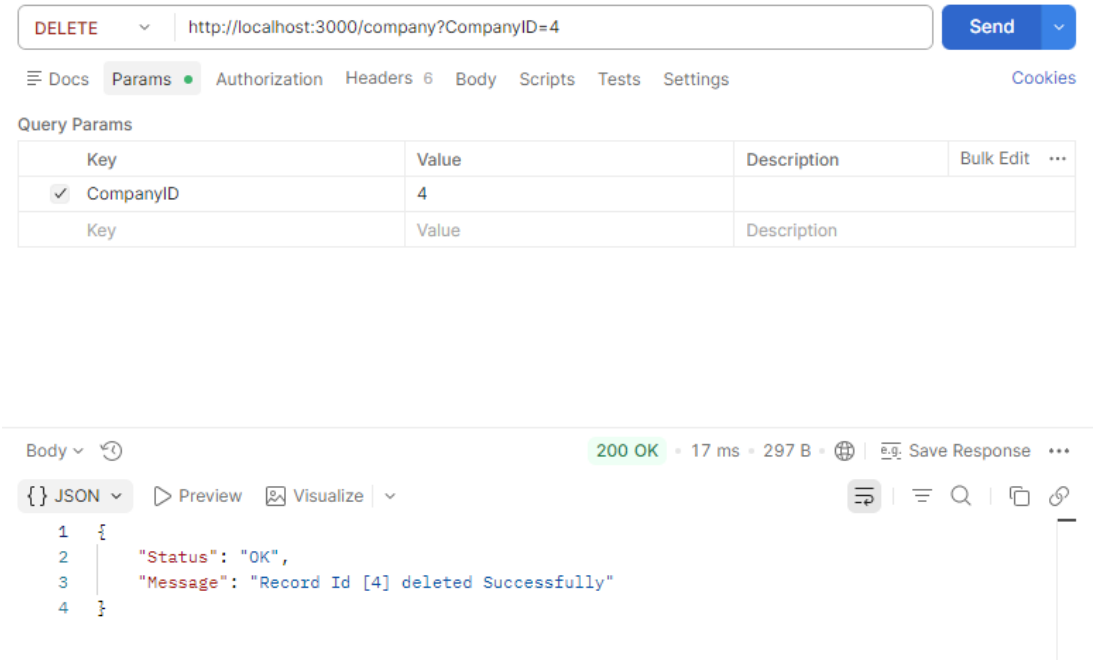


Figure 64: GET method for Company entity

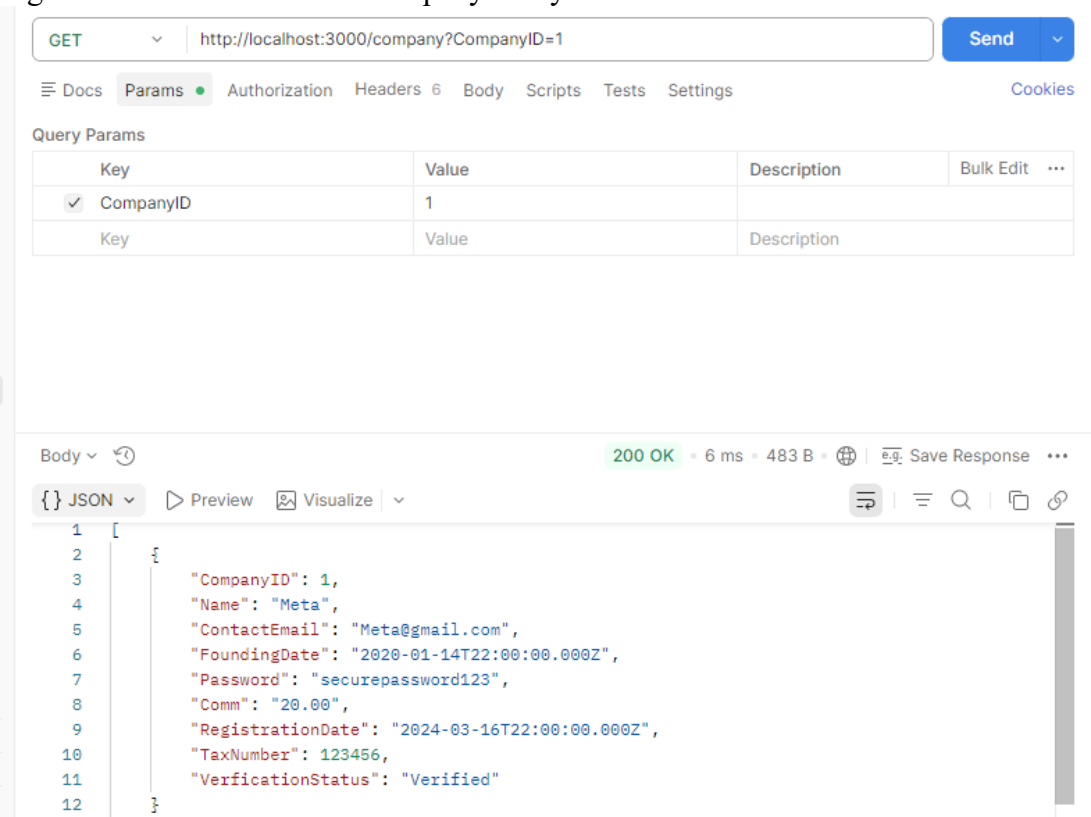
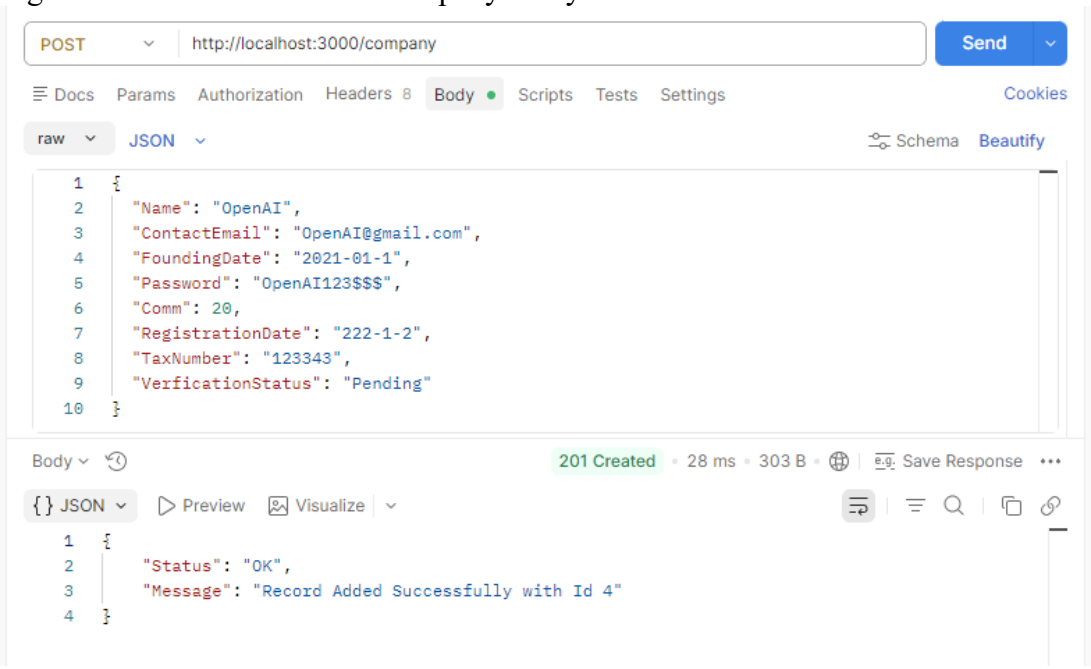


Figure 65: POST method for Company entity



Chapter 3: E-LMS Frontend Design

3.1 FOLDER STRUCTURE

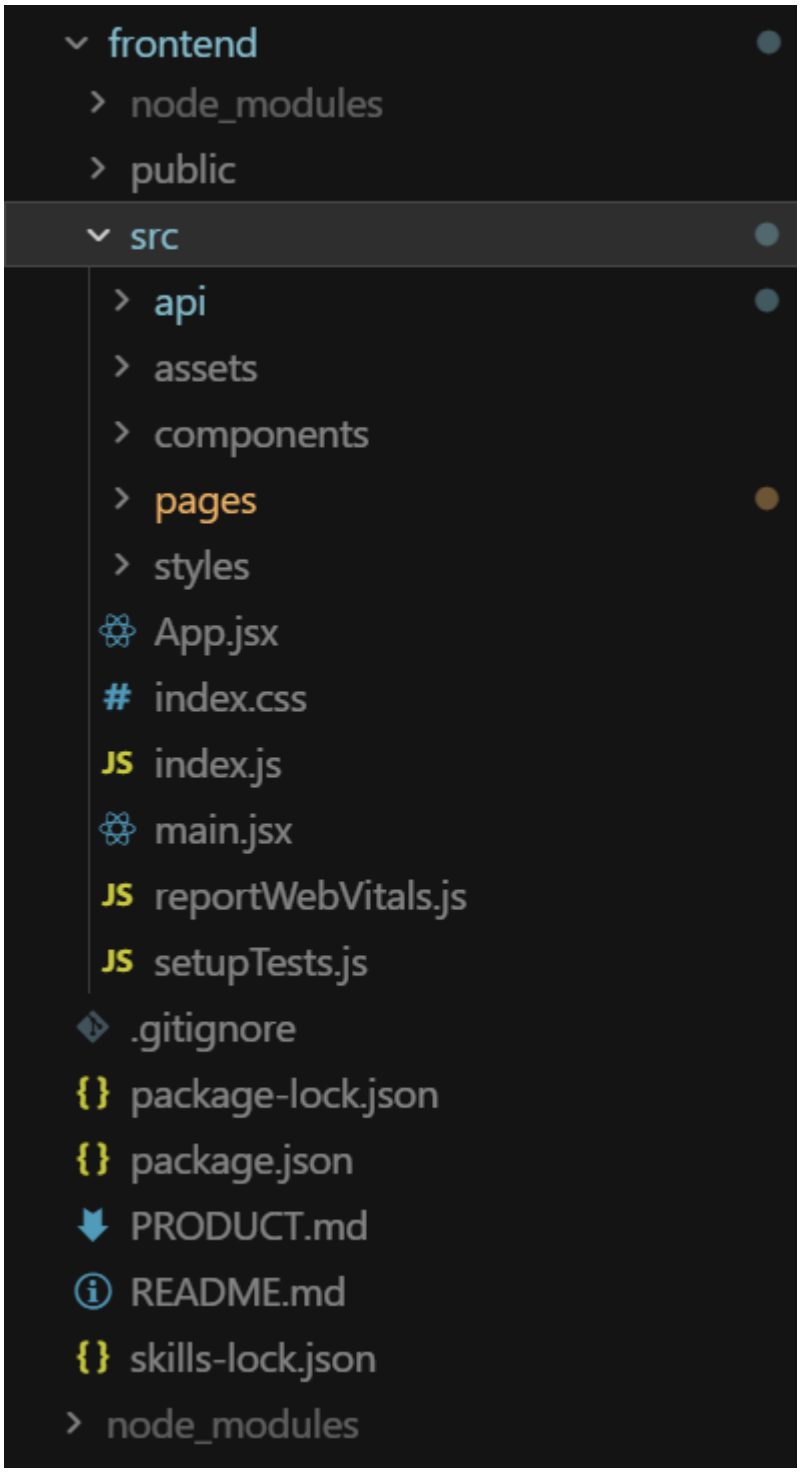


Figure 66: Frontend Folder Structure

3.2 CALLING APIS

Using React.js, we developed a web application that communicates with the previously developed Node.js/Express API to perform operations on the following entities: Users, Companies, Applications, Categories, Documents, Payments, and Support Tickets. All API calls are centralized in the frontend/src/api/ folder. A shared Axios instance is defined in client.js and configured with withCredentials: true so that the session cookie is automatically attached to every request, keeping the user authenticated across all calls. In the development environment, requests are sent as relative URLs (e.g. /user/login) and the React proxy forwards them to the backend running on port 3000. Each remaining file in the api/ folder covers one domain: auth.js handles login, logout, and registration; companies.js handles listing, fetching, registering, and verifying companies; applications.js handles submitting new applications, listing existing ones, updating their status, and fetching categories; documents.js handles creating document records linked to an application or client; payments.js handles submitting payments and support tickets; and admin.js handles fetching online users and the full user list for the admin dashboard. These functions are then called directly from inside useEffect hooks or form submit handlers in the page components, keeping all network logic separated from the UI.

3.3 FRONTEND DESIGN

- **useState:** Manages local, changing data across the app—form inputs, fetched data, loading states, errors, and UI flags (e.g., notifications, active items).
- **useEffect:** Handles side effects like fetching data on load, setting focus, triggering animations (IntersectionObserver), and syncing authentication state.
- **useMemo:** Optimizes performance by memoizing computed values such as filtered company lists and dashboard statistics.
- **useRef:** Stores references to DOM elements for actions like auto-focusing inputs and observing elements without re-rendering.

- **useNavigate:** Enables programmatic navigation (e.g., redirecting after login, logout, or form submission).
- **useParams:** Retrieves dynamic URL parameters (e.g., companyId, applicationId) to load relevant data.
- **useAuth (Custom Hook):** Combines state and effects to manage and sync authentication across the app using localStorage and browser events.

Backend port: 3000

Frontend port: 3001

This page intentionally left blank . . .

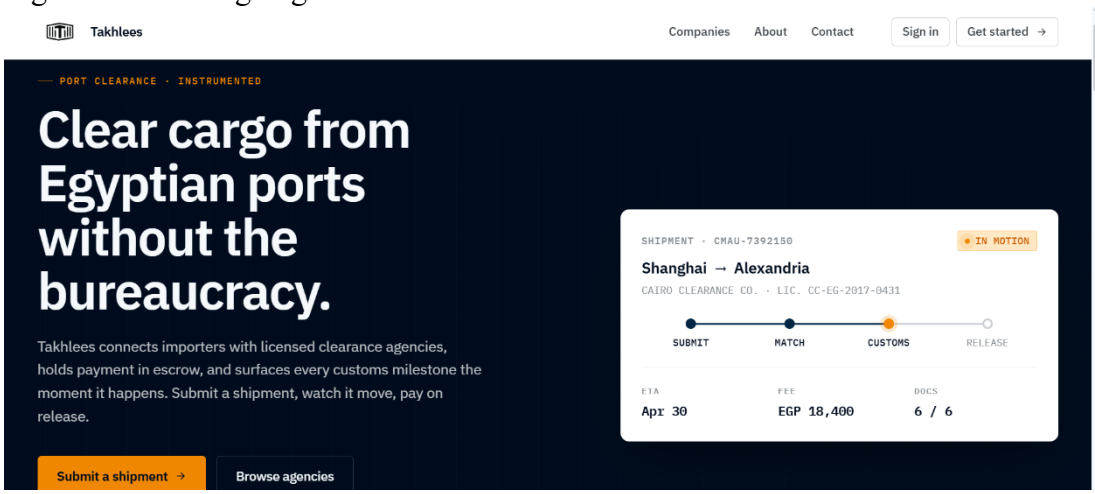
Chapter 4: Results

4.1 SYSTEM OVERVIEW

The following chapter shows the culmination of the Takhlees project, wherein the fully functional and integrated system is shown. It demonstrates the final results of the development stages, displays the working characteristics, refined user interfaces, and the successful implementation of the core port clearance workflows. The sections that follow give a detailed overview of the live application and assess the practical outcomes realized with regards to our initial goals.

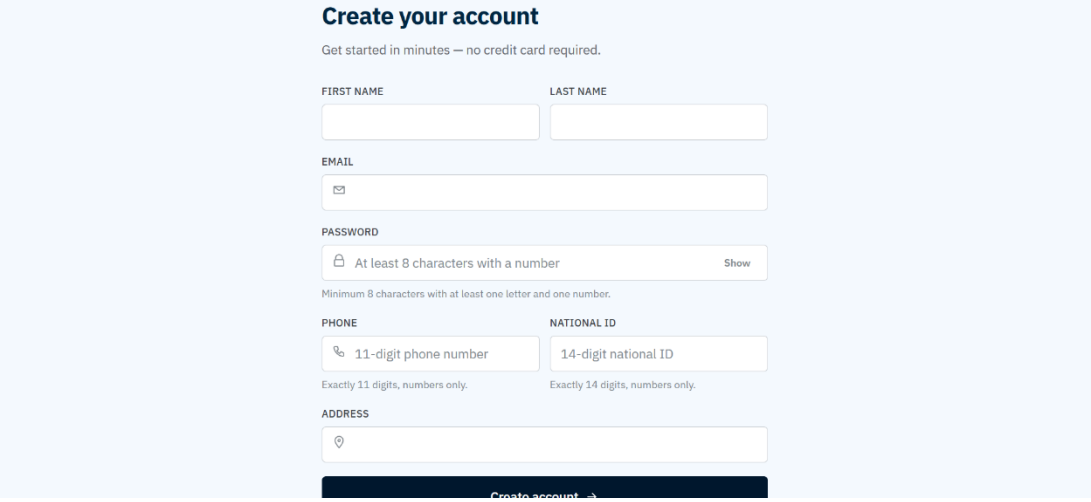
4.2 WEB APPLICATION INTERFACE

Figure 67: Landing Page



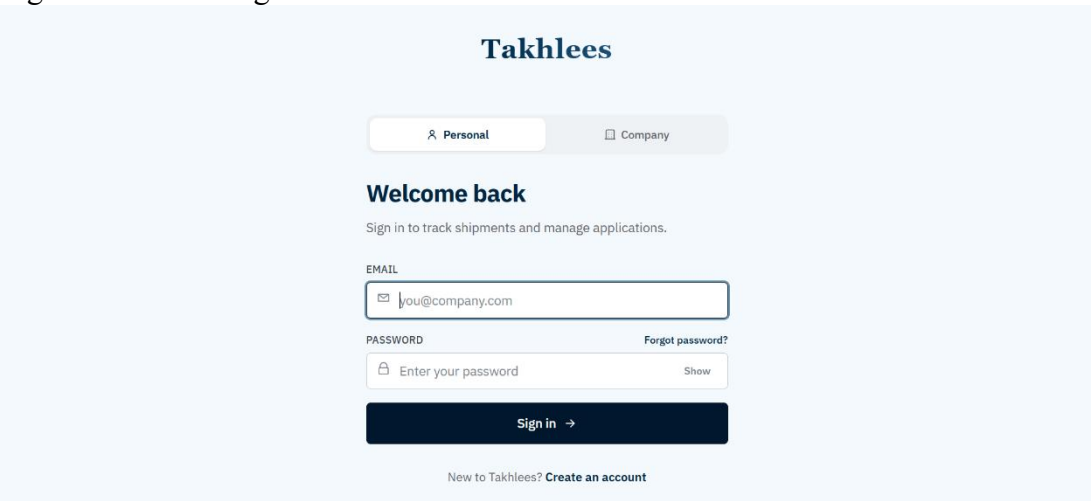
Landing Page: Public entry point that markets the platform and routes visitors to either submit a shipment or browse clearance agencies.

Figure 68: User Sign Up



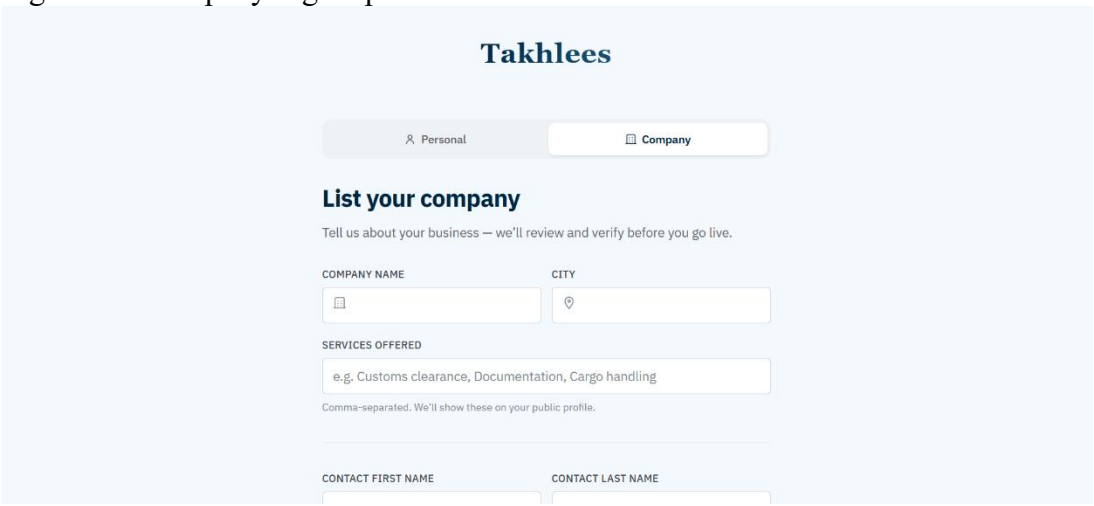
User Register (Personal): Validates and submits a new client signup (with phone, national ID, address) to create a Client user.

Figure 69: Client Sign In



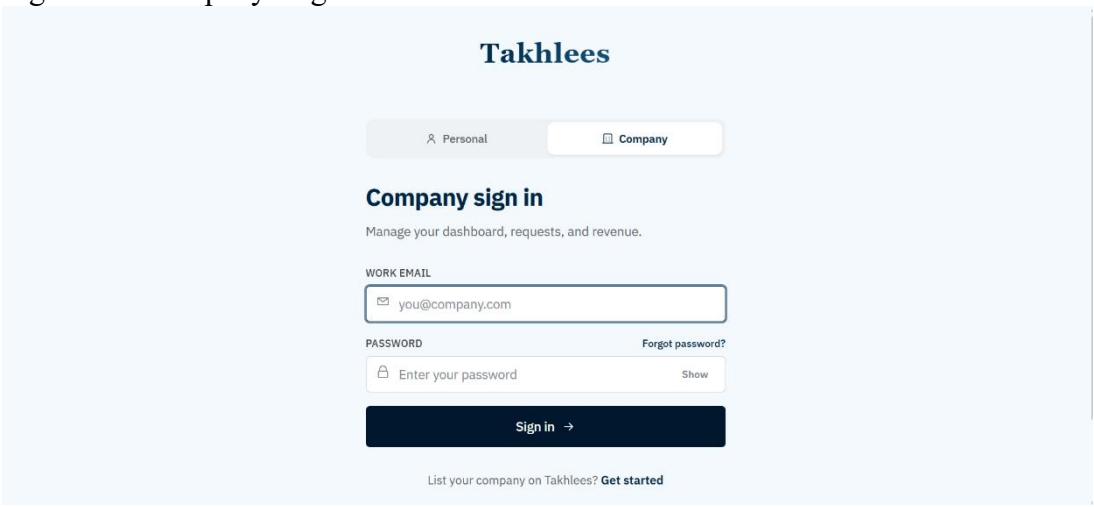
Client Sign In: Authenticates a client account against the backend and starts a session, redirecting to the client area on success.

Figure 70: Company Sign Up



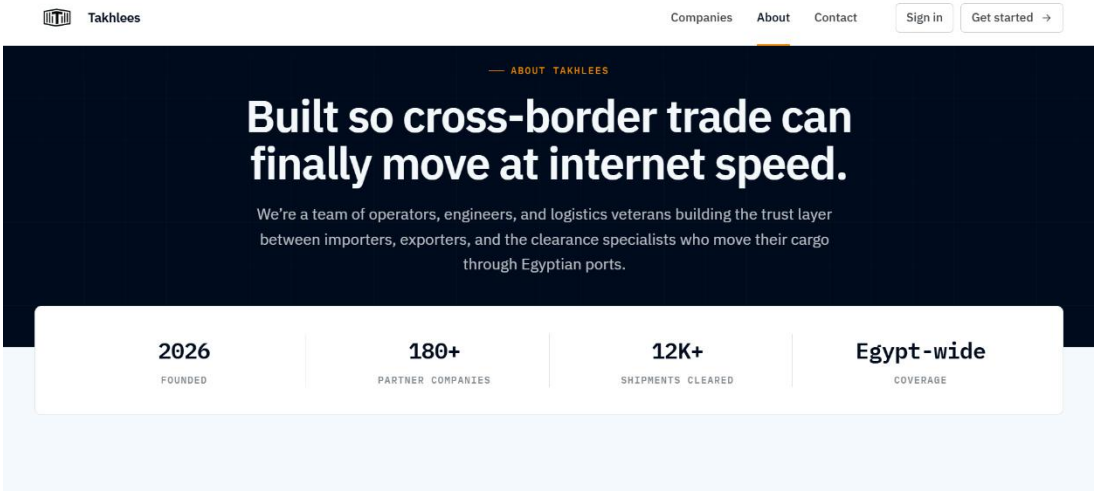
Company Register: Submits a new company application (business info + contact person) for admin verification before going live.

Figure 71: Company: Sign In



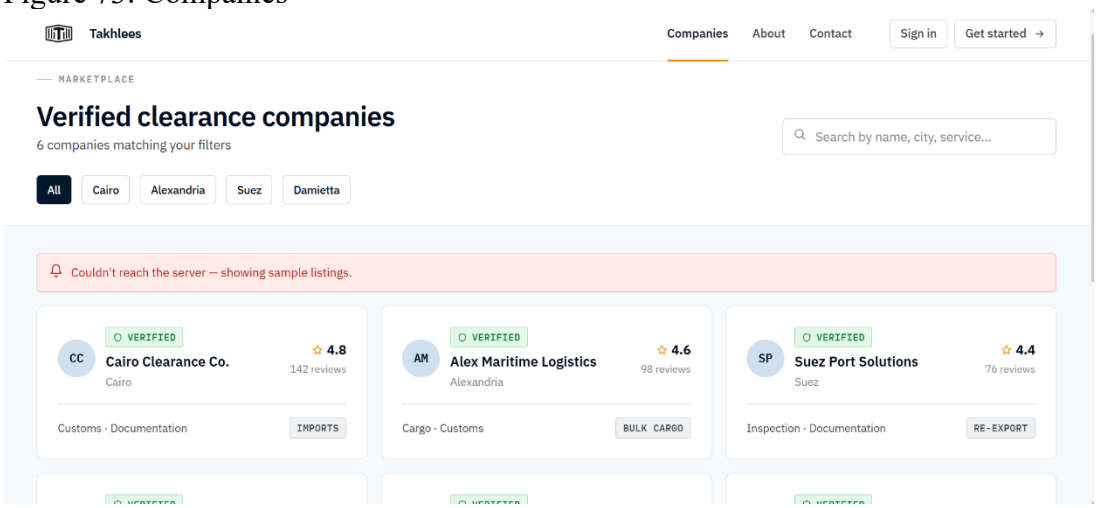
Company Sign In: Authenticates a company account and routes the user to the company dashboard.

Figure 72: About Us



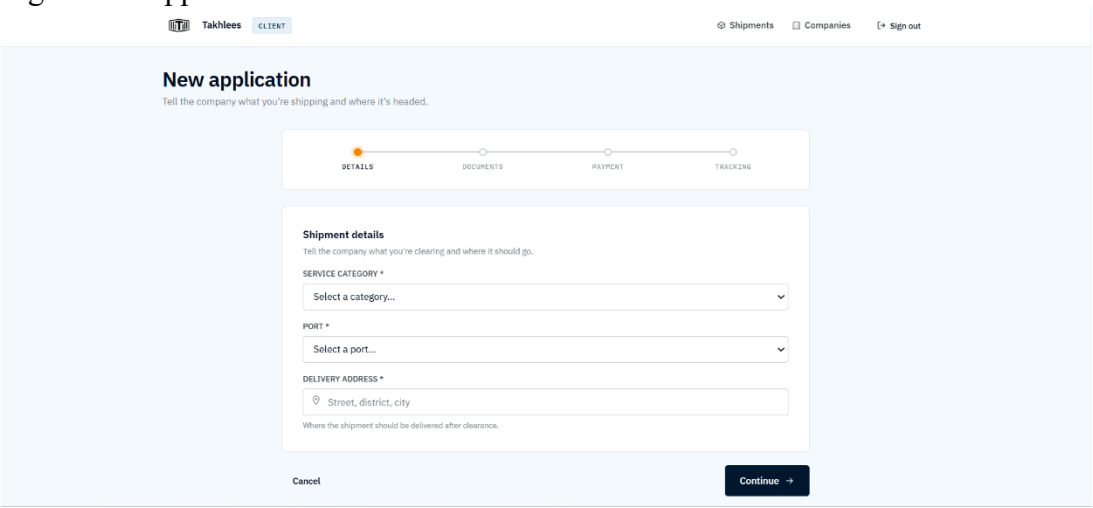
About Us: Static informational page explaining the company's purpose and credibility (no interactive logic).

Figure 73: Companies



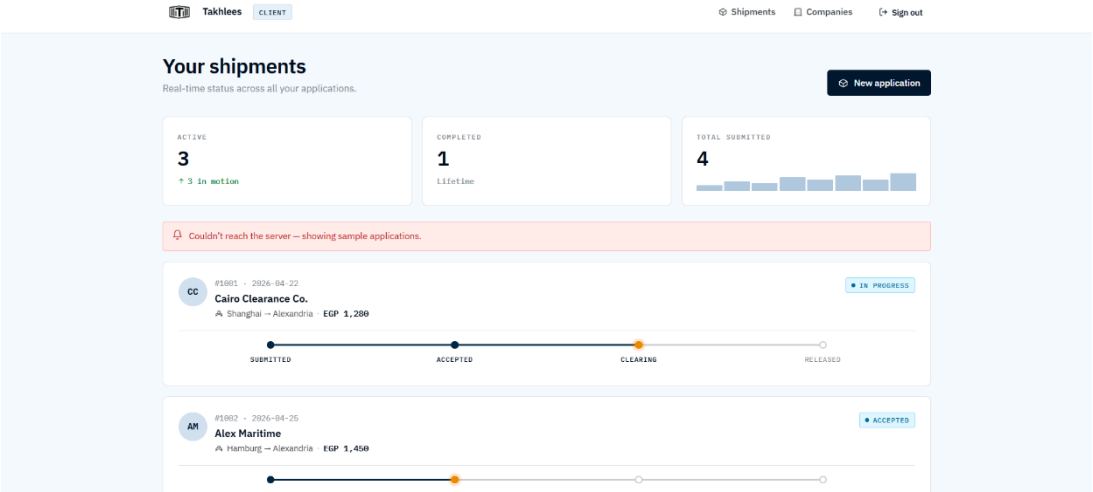
Companies: Fetches and lists verified clearance companies, lets users filter by city and search by name/service, and links to each company's profile.

Figure 74: Application



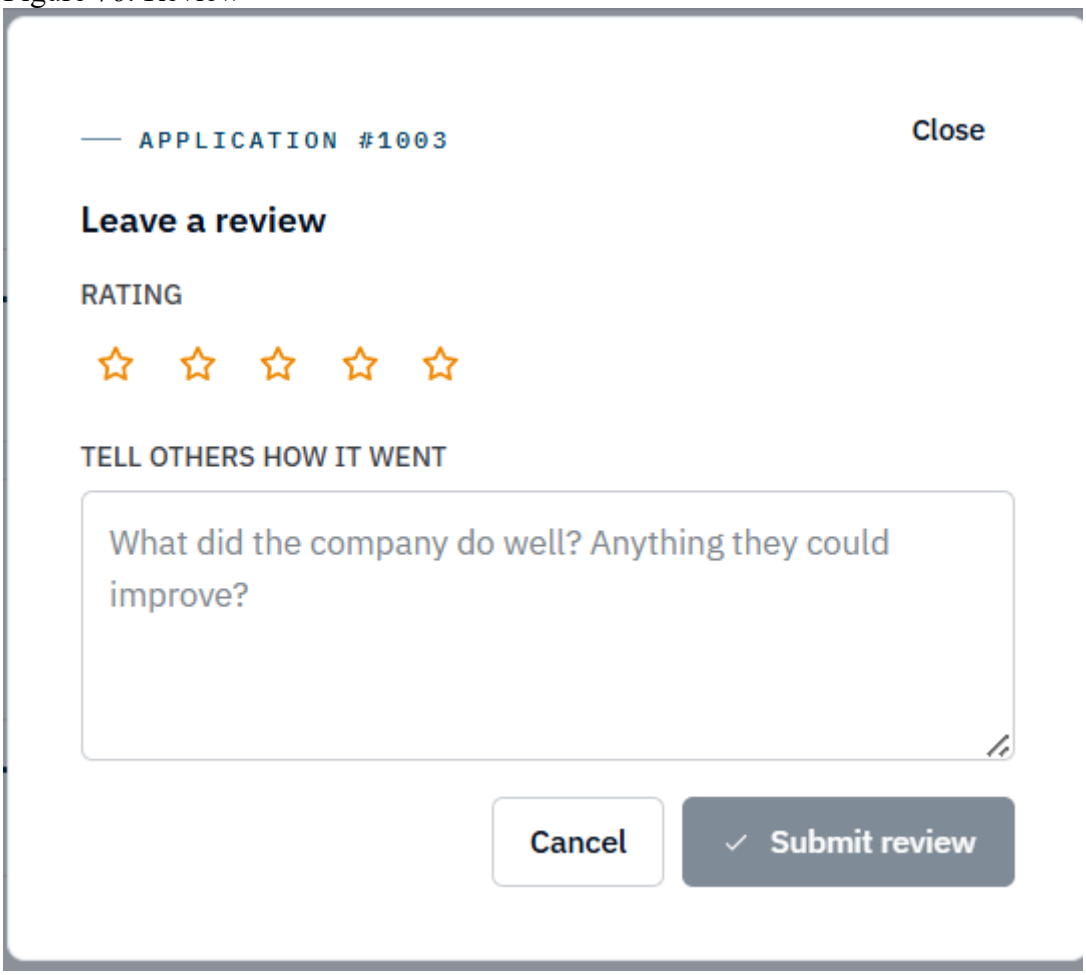
Fill Application: Multi-step form that collects shipment details, document uploads, and payment info, then submits a new clearance application.

Figure 75: Shipment Tracking



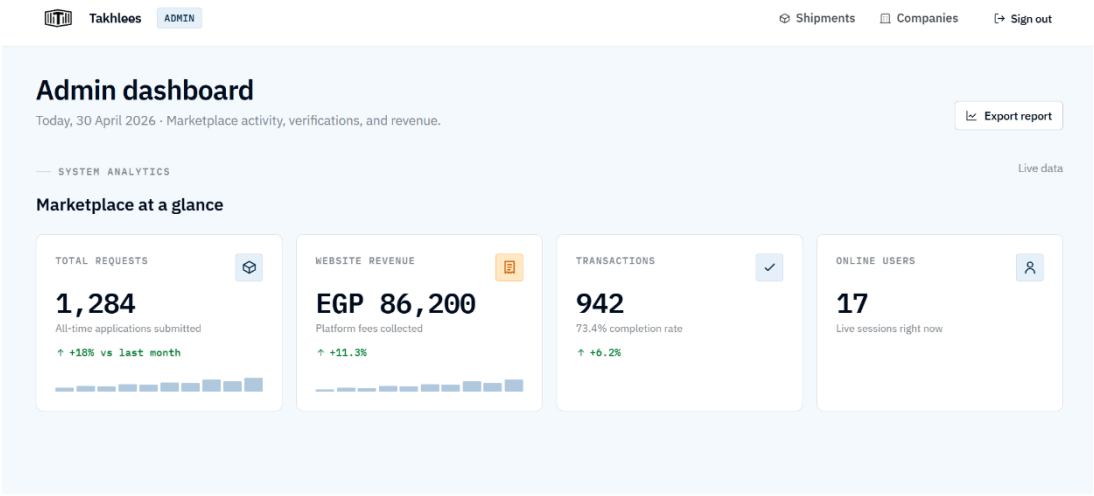
Client Tracking (Your Shipments): Lists the logged-in client's applications with live status, lets them open details, and triggers post-completion actions like leaving a review.

Figure 76: Review



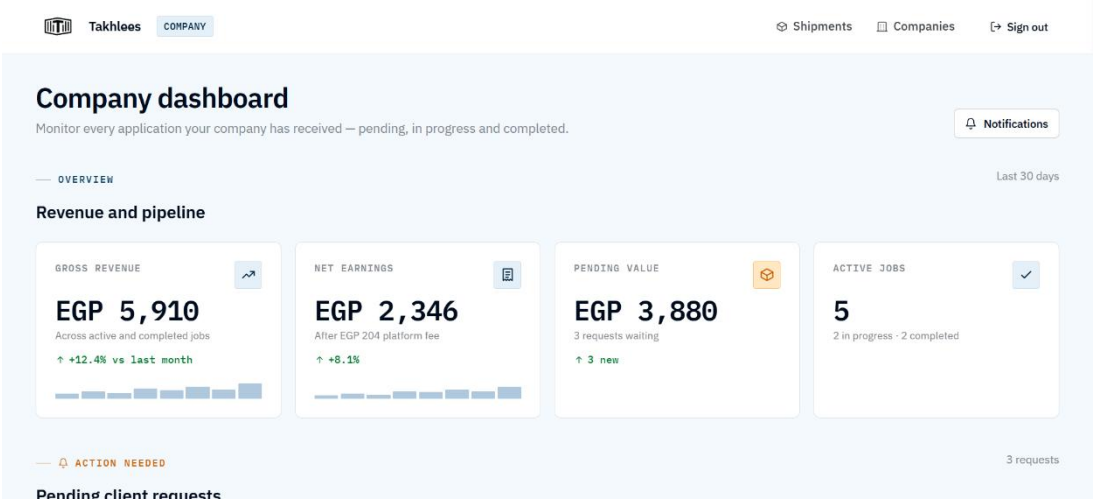
Leave a Review: Captures a star rating and comment for a completed application and posts it to the company's review record.

Figure 77: Admin Dashboard



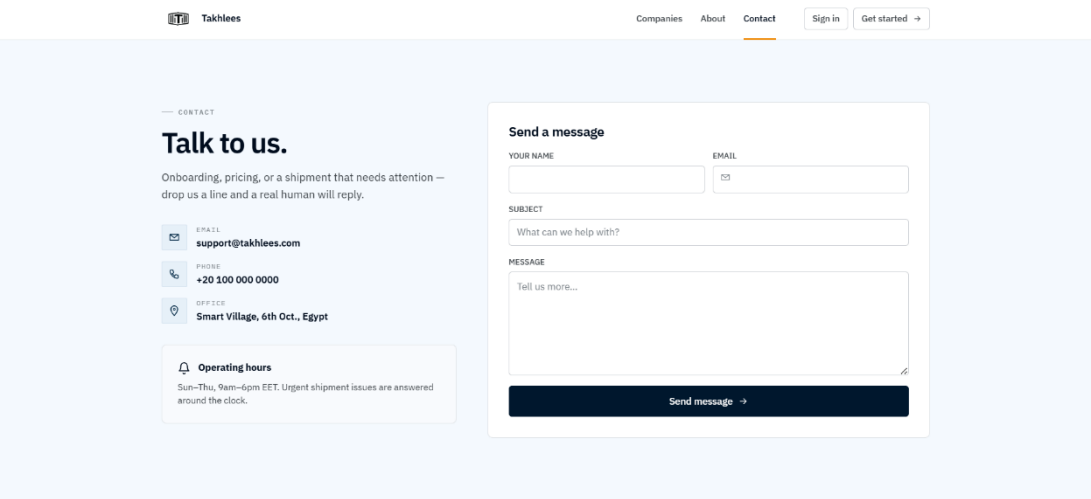
Admin Dashboard: Aggregates platform-wide metrics (requests, revenue, transactions, online users) for administrators to monitor activity.

Figure 78: Company Dashboard



Company Dashboard: Shows a company its own revenue, earnings, pending value, and active jobs, plus pending client requests it needs to act on.

Figure 79: Contact Us



Contact Us: Lets users and unauthenticated visitors send a support/inquiry message to the Takhlees team.

4.3 FUNCTIONAL RESULTS

Login & Logout: On the login page, the cursor auto-focuses on the email field. After entering credentials, the system validates inputs and checks them against the database. Invalid credentials return a generic error. If correct, a session is created with the user's ID and role, stored via cookies for authentication. Login attempts are rate-limited (5/min per IP). Logging out destroys the session and redirects to home.

Registration: Clients and companies register with required details. Inputs are validated (e.g., password rules, unique email/phone/ID). Client data is saved across two tables in a single transaction. Company registration follows a similar process.

Application Submission: Clients can browse companies, fill out an application, and submit it with a unique tracking number. The system validates and saves it, then redirects to payment.

Payment: Clients enter card details, and the system records the payment linked to the application before continuing the process.

Application Tracking: Clients can view all submitted applications and their statuses (e.g., Pending, In Progress, Completed).

Company Dashboard: Companies manage applications (accept/reject) and view stats like total pending and accepted requests.

Admin Dashboard: Admins verify companies, monitor users, and view platform analytics (users, applications, conversion rate).

Browse Companies (Public): Anyone can view verified companies and filter/search them instantly on the frontend.

Document Submission: Clients can attach documents (stored as metadata) with a "Pending" status for later verification.

Reviews: Clients can submit ratings and comments for completed applications, which can be managed via the API.

4.4 TESTING RESULTS

Table 2: Test Results

Feature	**Expected Result**	**Actual Result**
Login with correct email and password	User is logged in and redirected to their dashboard	Passed
Login with wrong password	Error message: "Invalid email or password"	Passed
Login with an email that does not exist	Error message: "Invalid email or password"	Passed
Login more than 5 times per minute (same IP)	Request is blocked with a rate limit error	Passed
Logout	Session destroyed and user redirected to home page	Passed
Register new client (valid data)	Account created and user redirected to login	Passed
Register with existing email	Error message: "Email already exists"	Passed
Register with existing phone number	Error message: "Phone number already exists"	Passed
Register with existing national ID	Error message: "National ID already exists"	Passed
Register with short password (<8 chars)	Error message: "Password must be at least 8 characters long"	Passed

Register password without numbers	Error: "Password must contain at least one letter and number"	Passed
Register new company (valid data)	Company created with "Pending" status	Passed
Register company with duplicate tax number	Error message: "Tax Number already exists"	Passed
Submit application (unique tracking no.)	Application saved and redirected to payment page	Passed
Submit application (duplicate tracking no.)	Error message: "Tracking Number already exists"	Passed
Complete payment	Payment recorded and linked to application	Passed
View tracking page	Applications listed with current status	Passed
Browse companies (not logged in)	Verified companies displayed publicly	Passed
Search companies by name	List filters in real time	Passed
Admin approves/rejects company	Company status updated in database	Passed
Company handles application	Status updated and dashboard refreshes	Passed
Admin views online users	Active users list displayed	Passed
Submit document	Document saved with "Pending" status	Passed
Submit review	Review saved with rating and comment	Passed
Access protected page (unauthenticated)	401 Unauthorized error	Passed

Access admin page as client	403 Forbidden error	Passed
Update profile (partial fields)	Only provided fields updated	Passed
Delete non-existent user	Error: "Record does not exist or already deleted"	Passed

4.5 DISCUSSION

Overall, Takhlees achieved its goal as a maritime clearance platform connecting clients, shipping companies, and admins in one web app. Users can find verified companies, submit applications with documents, make payments, and track status. Companies manage requests via dashboards, while admins oversee operations. All core features, from authentication to application and payment workflows, are fully functional.

Limitations:

1. No real payments: Payments are recorded but not processed (no gateway integration).
2. No file uploads: Only document metadata is stored, not actual files.
3. Weak password security (companies): Company passwords are not hashed.
4. No email notifications: No alerts for registration, updates, or payments.
5. No pagination: Large data loads may affect performance.
6. Partial analytics: Admin stats rely on hardcoded frontend data.
7. No real-time updates: Requires manual refresh to see changes.
8. No automated testing: All testing is manual.